AADL 语义规则

按 AS5506D 原文顺序重新梳理

编排：从本地 `AS5506D-1791452205139.pdf` 的正文、图示及本卷所含附录逐章提炼，共 705 条。条目依首次正式定义的章节及段落顺序排列；后续章节的补充条件并入对应条目并保留出处。编号为“标准小节–本节序号”，每节从 01 开始，不沿用旧版的静态、动态及主题编号。

收录：以合法模型能够判断的结构、属性求值及运行结果为单位，覆盖主标准的软件组件、通信、执行平台、模式和系统运行语义，以及本卷属性、运行服务和 Data Sets。纯概念介绍、语法、命名检查及实现文档义务不单独列项；标准存在不一致时，仅表述无争议条件下的结果。

粒度：一个独立可判断的结果对应一条规则，适用条件、例外与默认值随条保留。能够分别出错的状态变化、值变化和计时结果分别编号；共同构成同一判断的条件不单独拆条。同一结果在多处出现时，只编号一次并合列依据。

阅读：每条包含简短标题、适用条件与结果、原文条款及页码。页码均为标准页码。目录只列承载独立条目的章节；后文仅重复或补充已有结果的内容已经合并，只有引用而未在本卷给出正文的独立附录不在此扩展。

目录

第 4 章　组件、包与附录 **4**

4.1　AADL 规范 4
4.2　包 4
4.3　组件类型 4
4.4　组件实现 5
4.5　子组件 6
4.6　抽象组件 8
4.7　原型 8
4.8　附录子句与附录库 9

第 5 章　软件组件 **10**

5.1　数据 10
5.1.1　共享数据访问的运行时支持 12
5.2　子程序 13
5.3　子程序组 15
5.4　线程 16
5.4.1　线程状态与动作 17
5.4.2　线程分派 20
5.4.3　线程调度与执行 21
5.4.4　线程故障处理 23
5.4.5　线程内部模式 25
5.4.6　同步执行语义 25
5.4.7　异步执行语义 26

5.4.8 线程运行时支持 . . . 26
5.5 线程组 . . . 27
5.6 进程 . . . 28

第 6 章 执行平台组件 **30**
6.1 处理器 . . . 30
6.2 虚拟处理器 . . . 34
6.3 存储器 . . . 35
6.4 总线 . . . 36
6.5 虚拟总线 . . . 37
6.6 设备 . . . 37

第 7 章 组合组件 **39**
7.1 系统 . . . 39

第 8 章 特征 **40**
8.2 特征组 . . . 40
8.3 端口 . . . 40
8.3.1 端口类别 . . . 40
8.3.2 输入与输出时刻 . . . 42
8.3.3 队列与扇出策略 . . . 44
8.3.4 事件与子程序 . . . 47
8.3.5 端口运行时服务 . . . 47
8.4 子程序与子程序组访问 . . . 49
8.5 子程序参数 . . . 49
8.6 数据访问 . . . 50
8.7 总线访问 . . . 51
8.8 虚拟总线访问 . . . 51
8.9 内部特征与处理器代理特征 . . . 51

第 9 章 连接 **52**
9.1 特征连接 . . . 52
9.2 端口连接 . . . 53
9.2.1 端口连接特征 . . . 54
9.2.2 端口连接拓扑 . . . 55
9.2.3 组件数组与特征数组的连接模式 . . . 56
9.2.4 连接传输的时机 . . . 58
9.2.5 采样、即时与延迟数据端口通信 . . . 59
9.2.6 语义端口连接与端口队列 . . . 60
9.3 参数连接 . . . 62
9.4 访问连接 . . . 62
9.5 特征组连接 . . . 63

第 10 章　流 **64**
10.1　流规格 64
10.2　流实现 64
10.3　端到端流 65

第 11 章　属性 **67**
11.1.1　属性类型 67
11.1.2　属性定义 67
11.1.3　属性常量 68
11.3　属性关联 68
11.4　属性表达式 70

第 12 章　模式 **73**

第 13 章　系统运行语义 **75**
13.1　系统实例 75
13.2　应用软件的执行平台绑定 75
13.3　系统启动 76
13.4　正常系统运行 78
13.5　系统运行模式 79
13.6　系统运行模式转换 79
13.7　系统范围的故障处理、关闭与重启 85

第 14 章　分层系统的建模 **87**

第 15 章　词法元素 **88**
15.4.1　十进制字面量 88
15.4.2　基数字面量 88

附录 A　预声明属性集与运行时服务 **89**
A.1　预声明部署属性 89
A.2　预声明线程属性 92
A.3　预声明时间属性 94
A.4　预声明通信属性 96
A.5　预声明内存属性 97
A.6　预声明编程属性 100
A.8　项目属性集 101

附录文档 F（本卷收录）　Data Sets **102**
F.1　Data Sets 的定义与关联 102

第 4 章　组件、包与附录

4.1　AADL 规范

4.1-01　系统实例根

系统实例由作为根的 system implementation 确定；对该实现的引用标识待建立的系统实例。

依据： 4.1(5)（第 26 页）。

4.2　包

4.2-01　公私实现声明合一

同一组件实现同时在包的 public 和 private 部分声明时，两份声明共同表示同一个实现；外部可见部分只公开属性和模式，内部子组件与连接由 private 部分补全。若该实现为扩展，extends 及其原型绑定在 public 声明中确定，private 声明完成该扩展。

依据： 4.2(9)–(10)（第 30 页）。

4.2-02　包属性的关联对象

包 properties 部分的属性关联属于包声明本身；包内组件实例化不使该关联自动变成组件实例的属性。

依据： 4.2(13)（第 30 页）。

4.2-03　分离包声明独立保留属性值

同一个包的 public 和 private 分别作为包声明保存时，两份声明可以为同一属性保留不同的关联值。

依据： 4.2(13)（第 30 页）。

4.3　组件类型

4.3-01　类型模式对所有实现有效

组件类型声明的模式和模式转换对该类型的每个实现共同有效，包括未通过 extends 扩展其他实现的普通实现。类型模式不是每个实现可任意另选的一套模式。

依据： 4.3(3)（第 31 页）；4.3(13)（第 34 页）。

4.3-02　类型扩展保留并增加声明

类型扩展的有效声明包含祖先类型继承的特征、流规格、模式、模式转换、原型和属性，以及该扩展新增的对应声明；继承元素不会仅因创建后代类型而消失。

依据： 4.3(4)（第 31 页）；4.3(17)（第 35 页）。

4.3-03　类型特征细化补全分类器

类型扩展细化不完整特征声明时，后代的该特征使用细化后补全的分类器；细化改变同一继承特征，不另增一个同名特征。

依据： 4.3(4)（第 31 页）；4.3(17)（第 35 页）。

4.3-04　类型细化改变元素属性值

类型扩展在继承特征或流规格上关联新的属性值时，后代元素的属性按该细化后的关联求值；该变化不另创建特征或流规格。

依据： 4.3(4)（第 31 页）；4.3(17)（第 35 页）。

4.3-05　所需模式的转换由容器驱动

组件类型声明 requires modes 时，组件使用由容器提供的模式；这些模式的转换行为由容器决定，组件不为这些所需模式另作独立转换决定。

依据： 4.3(14)（第 34 页）。

4.3-06　原型绑定保持既有实参

通过类型或实现扩展、特征或子组件引用提供的原型绑定用于补全模板；已经绑定的原型不能再次绑定。部分绑定之后，后续引用与扩展只能为剩余未绑定原型提供实参。

依据： 4.3(16)–(18)（第 34–35 页）；4.4(11)（第 40 页）；4.7(4)（第 55 页）。

4.3-07　抽象类型的具体化类别

abstract 组件类型被扩展为具体组件类别时，后代采用该具体类别解释，并保留可继承的祖先声明；abstract 祖先不因此作为额外包含实例加入后代。

依据： 4.3(17)（第 35 页）。

4.3-08　附录继承由附录决定

类型或实现扩展是否继承祖先的附录声明，由该附录自身定义；核心语言元素的继承不自动使附录内容继承。

依据： 4.3(17)（第 35 页）；4.4(10)（第 40 页）；4.8(11)（第 58 页）。

4.4　组件实现

4.4-01　无实现类型形成叶节点

组件类型没有任何实现时，按该类型建立的组件是系统包含层次中的叶节点；叶节点保留类型规定的接口和属性，不生成未声明的内部子组件。

依据： 4.4(2)（第 36 页）。

4.4-02　实现扩展继承实现内部声明

实现扩展的有效内部声明保留祖先实现的原型、子组件、连接、调用序列、流、模式、模式转换及属性，细化部分按其细化结果解释。

依据： 4.4(3)–(5)（第 36 页）；4.4(10)（第 40 页）。

4.4-03　实现取得所属类型的接口

组件实现取得所属组件类型及其类型祖先的特征与属性；实现的内部 extends 链不会替代这一组件类型来源。

依据： 4.4(5)（第 36 页）；4.4(10)（第 40 页）。

4.4-04　实现细化补充有效分类器

实现扩展为继承的原型或子组件补充分类器后，该原有元素使用补充后的分类器；补充分类器不另增一个同名元素。

依据： 4.4(11)（第 40 页）。

4.4-05　实现级细化关联元素属性

实现扩展可为继承的原型、子组件、连接、流、模式或所属类型的特征关联新属性；相应元素按这些有效关联取值。对类型特征的实现级细化只改变属性关联，不重定义其分类器。

依据： 4.4(11)（第 40 页）。

4.4-06　实现扩展增加内部元素

实现扩展新增的原型、子组件、连接、调用序列、流序列、模式和附录声明加入该后代实现的有效结构，而不改变祖先实现本身的结构。

依据： 4.4(12)（第 41 页）。

4.4-07　继承元素随扩展实现实例化

建立组件实现扩展的实例时，本地声明和继承而来的子组件、特征、连接、调用序列、流序列、模式均按有效声明进入实例；不只实例化扩展中新写出的元素。

依据： 4.4(13)（第 41 页）。

4.5　子组件

4.5-01　数组实现表按末维优先映射

为子组件数组逐元素给出实现列表时，各元素采用列表对应的实现；多维数组先递增最后一维下标，再推进此前维度。各实现共享所声明的组件类型。

依据： 4.5(L9)–(L11)（第 45 页）；4.5(14)（第 46 页）。

4.5-02　数组索引确定实例集合

大小已定的子组件数组按各维 1 到该维大小的全部索引组合展开，每组索引对应一个独立子组件实例；大小未定的不完整数组在不完整系统实例中按一个元素处理，补定原先未定维度后按所定大小展开。

依据： 4.5(L10)（第 45 页）；4.5(13)（第 46 页）。

4.5-03　每个父实例分别展开子实例

组件实现实例化时，其包含的子组件同时在该父实例下实例化，并按所选实现继续向下展开；不同父实例下同一子组件声明产生不同的子实例。

依据： 4.5(6)（第 45 页）。

4.5-04　子组件采用已绑定的原型实参

子组件引用组件原型时，其实例使用原型绑定的实际组件类别和分类器；同一原型的多处引用解析到同一个已供应分类器，而不是各自重新选择。

依据： 4.5(8)（第 45 页）；4.7(2)（第 55 页）。

4.5-05　原型实参向下层传递

下层子组件的原型绑定引用容器原型时，下层取得容器原型已经绑定的实际分类器或特征；该传递可沿组件层次继续进行。

依据： 4.5(9)（第 45 页）。

4.5-06　带绑定引用构成未命名扩展

分类器引用携带具体原型绑定时，按被引用分类器的一个未命名扩展解释；不同引用形成的未命名扩展不因共同使用原分类器而互为扩展。

依据： 4.5(10)（第 45 页）。

4.5-07　子组件在所列容器模式中活动

普通子组件声明的 component_in_modes 给出其活动的容器模式集合；未给该子句时，该子组件在容器全部模式中活动。

依据： 4.5(11)（第 45–46 页）；4.5(11)（第 46 页）；12(11)（第 216 页）。

4.5-08　细化替换子组件活动模式集合

子组件细化带 component_in_modes 时，以新的集合替换原活动模式子集；细化不带该子句时，按 4.5(11) 解释为在全部模式中活动，不保留旧子集。

依据： 4.5(11)（第 45–46 页）。

4.5-09　子组件细化覆盖属性

子组件细化中新关联的属性值覆盖被细化声明中的相应值；模式相关的新值只在其对应模式条件下参与求值。

依据： 4.5(12)（第 46 页）。

4.5-10　数组声明属性覆盖每个元素

直接关联到子组件数组声明的属性值适用于该数组的每一个元素。

依据： 4.5(15)（第 46 页）。

4.5-11　数组元素属性可分别覆盖

容器实现中的 contained property association 分别作用于指定的数组元素或元素子集；路径还有后续部分时，对各选中元素分别解析余下路径，并将属性值关联到相应终点。未选中的元素或目标继续采用其原有有效值。

依据： 4.5(15)（第 46 页）；11.3(10)（第 205 页）。

4.5-12　单一实现允许自动选择

子组件只引用类型且该类型仅有一个实现时，处理工具可以选择这个未显式命名的唯一实现，产生完整实例；此选择属于许可，不是必须行为。类型存在多个实现时，不能由该许可任意选择一个实现，所用实现须显式指明。

依据： 4.5(16)（第 46 页）。

4.6　抽象组件

4.6-01　具体化后忽略不适用属性

abstract 组件被具体化后，不适用于所得具体类别的属性被忽略，不再作为该具体组件的有效属性；工具可以报告被忽略的属性。

依据： 4.6(2)（第 50 页）。

4.7　原型

4.7-01　列表原型提供数组分类器

以方括号声明、接受分类器列表的组件原型用于子组件数组时，绑定列表为该数组各元素提供对应分类器。

依据： 4.7(L9)（第 54 页）。

4.7-02　特征原型补全具体特征

抽象特征引用特征原型时，原型绑定提供的具体特征决定该位置的实际特征类别、方向和分类器；原型本身不形成额外特征实例。

依据： 4.7(3)（第 55 页）；8.1(3)（第 123 页）。

4.7-03　特征组原型确定组接口

特征组引用特征组类型原型时，绑定的实际特征组类型决定该组的接口及成员。

依据： 4.7(3)（第 55 页）。

4.8 附录子句与附录库

4.8-01 附录子句按模式生效

附录子句带 in_modes 时，其附录语句只适用于列出的模式；不带 in_modes 时适用于全部模式。

依据： 4.8(5)（第 57 页）。

4.8-02 核心处理可不解释附录内容

仅符合核心 AADL 的处理方法可以接受含附录的模型而不处理附录子句或附录库内部内容；不能据核心模型处理成功推断该附录行为已经被执行或分析。

依据： 4.8(7)（第 58 页）。

第 5 章　软件组件

5-01　处理后的源程序表示可以被接受

软件源文本可以是编译等处理产生的中间产品，例如目标文件；处理方法允许把这种产品视为符合所用语言的源文本表示，不只接收未处理的源代码文本。

依据： 5(2)（第 59 页）；A.6 Source_Language（第 271 页）。

5.1　数据

5.1-01　源数据类型名的映射

Type_Source_Name 指定 data 分类器所对应的源语言类型名称；该类型名映射不替代数据变量的名称映射。

依据： 5.1(5)（第 61 页）；A.6 Type_Source_Name（第 272 页）。

5.1-02　源数据变量名的映射

data 子组件的 Source_Name 指定对应的静态或局部源变量名称；它标识源变量，不改变模型中的声明名称。

依据： 5.1(5)（第 61 页）；A.6 Source_Name（第 271 页）。

5.1-03　数据访问方法的隐式实例

以 data type 的 provides subprogram access 表示隐式子程序实例时，该实例属于调用所在进程，同一进程内使用同一个实例；它可以作为可调用的数据访问方法。采用这一隐式实例解释的实现允许该 feature 不再连接到显式子程序子组件。

依据： 5.1(7)（第 61 页）；5.1.1(24)（第 63 页）。

5.1-04　数据内部元素的访问方式

在适用访问权限允许的范围内，已取得数据对象访问的应用代码可以直接访问、修改其内部元素，也可以通过该 data type 提供的子程序操作；核心语言不为这些内部元素另外施加源语言式的可见性级别。

依据： 5.1(8)（第 61 页）。

5.1-05　数据实现的字段映射

data implementation 中的 data 子组件分别表示源数据结构中的字段或实例变量。

依据： 5.1(9)（第 61 页）。

5.1-06　未给实现时不补造内部字段

只引用 data type 且未给 data implementation 时，保留该类型和属性，不自动生成未建模的源类型字段或内部结构。

依据： 5.1(10)（第 61 页）；5.1.1(22)（第 63 页）。

5.1-07　数据声明分别生成实例

每个 data 子组件声明产生自己的数据实例；引用相同分类器不把这些声明合并成一个对象。

依据： 5.1(12)（第 61 页）。

5.1-08　独立数据实例的写入隔离

对不同 data 子组件声明形成的独立实例写入时，修改的是被选中的实例；不能仅因为分类器或源数据映射相同，就同时改变另一个独立实例的值。

依据： 5.1(12)（第 61 页）；5.1.1(26)（第 63 页）。

5.1-09　静态数据跨分派保持

不位于 subprogram 内的 data 子组件具有静态存储，其值跨线程分派保持，除非执行了改变该值的操作；subprogram 内的 data 子组件属于局部数据，不按此静态持久性解释。

依据： 5.1(12)–(13)（第 61 页）；5.2(7)（第 68 页）。

5.1-10　子程序局部数据不跨调用保持

subprogram 内声明的 data 子组件属于本次调用的局部数据，调用结束后不作为该子程序的跨调用持久状态。

依据： 5.1(13)（第 61 页）；5.2(7)（第 68 页）。

5.1-11　数据类型的实例存储量

data type 上的 Data_Size 给出容纳一个该类型实例值所需的最大空间；即使没有 data implementation，仍可按该值解释实例的内存需求。

依据： 5.1(14)（第 61 页）；A.5 Data_Size（第 265 页）。

5.1-12　共享访问解析到同一对象

多个 requires data access 沿访问连接解析到同一 data 子组件或同一提供访问端时，各使用者访问同一共享对象，不为每个需求访问复制对象。

依据： 5.1(15)（第 61 页）；8.6(3)–(4)（第 146 页）。

5.1-13　只读数据对象保持常量

数据组件本身的 Access_Right 为 read_only 时，该实例表示值不可改变的常量；不同使用者访问同一实例时观察同一个不变值。

依据： 5.1(15)（第 61 页）。

5.1-14　临界区入口和出口的资源操作

线程进入受 Concurrency_Control_Protocol 管理的数据临界区时执行 Get_Resource，退出该临界区时执行对应 Release_Resource。

依据： 5.1(16)（第 62 页）；5.1.1(20)（第 62 页）；A.9(12)（第 281 页）。

5.1-15　嵌套资源操作成对返回

同一线程嵌套访问多个受控数据组件时，各对象的 Get_Resource 与 Release_Resource 成对嵌套，内层临界区先于外层退出。

依据： 5.1(16)（第 62 页）。

5.1-16　数据方法使用对象并发协议

通过数据类型提供的方法访问共享 data 时，这些方法的执行也受该 data 的 Concurrency_Control_Protocol 协调。

依据： 5.1(17)（第 62 页）。

5.1-17　AADL 数据数组不是一个源数组对象

data 子组件数组是多个独立 data 子组件声明的简写，其展开结果是多个数据实例；若需表示一个源程序数组对象，应由该数据类型的数据建模信息描述，而不能只据 AADL 子组件数组把这些实例合并为一个源数组。

依据： 5.1(19)（第 62 页）。

5.1.1　共享数据访问的运行时支持

5.1.1-01　未关联源文本时的数据类型解析

data type 未关联源文本时，对应类型须在相关源码最外层作用域可见；默认由模型数据类型名按源语言规定映射，也可用 Type_Source_Name 显式指定。

依据： 5.1.1(21)（第 62–63 页）。

5.1.1-02　数据赋值故障的归属

实现使用运行时检查判断实际赋值是否允许时，检查产生的运行时故障归属于包含赋值数据源的线程；不能把故障归给仅接收该值的另一线程。

依据： 5.1.1(25)（第 63 页）。

5.1.1-03　重复源数据映射的二进制复制

实现允许同一地址空间中的多个静态 data 声明映射到同一份源数据时，二进制映像须为这些声明复制数据；实现也允许限制这种重复映射。

依据： 5.1.1(26)（第 63 页）。

5.1.1-04　同时获取多资源的无死锁保证

实现支持一次 Get_Resource 同时锁定多个资源时，该服务的多资源获取机制须保证不会死锁。只支持单资源获取时，避免不安全调用次序的责任在调用方。

依据： 5.1.1(27)–(28)（第 63 页）。

5.1.1-05　同时获取多资源的无饥饿保证

实现支持一次 Get_Resource 同时锁定多个资源时，该服务须保证资源获取不会饥饿；不能将此保证无条件套用于仅支持单资源获取的任意调用次序。

依据： 5.1.1(27)–(28)（第 63 页）。

5.2　子程序

5.2-01　分类器调用的隐式实例

调用直接引用子程序分类器而没有显式子程序子组件时，按所在进程内的隐式子程序实例解释。

依据： 5.2(4)（第 65 页）。

5.2-02　单次子程序调用的顺序执行

一次子程序调用执行源程序规定的顺序代码序列，后续动作使用本次调用先行动作已经产生的结果。

依据： 5.2(5)（第 68 页）。

5.2-03　调用进入时提供输入参数

子程序调用开始时，in 与 in out 参数按参数连接向被调用子程序提供本次输入值。

依据： 5.2(6)（第 68 页）；5.2(15)（第 69 页）；9.3(2)（第 168 页）。

5.2-04　调用返回时交付输出参数

子程序调用返回时，out 与 in out 参数的本次结果按参数连接交给相应目的端。

依据： 5.2(6)（第 68 页）；5.2(15)（第 69 页）；9.3(2)（第 168 页）；9.3(5)（第 170 页）。

5.2-05　子程序持久状态由外部数据承载

子程序需要跨调用状态时，通过 requires data access 访问外部持久 data；后续调用通过同一访问连接读取该共享对象的当前值，并遵循其并发控制协议。

依据： 5.2(8)（第 68 页）。

5.2-06　子程序内显式发送事件

执行子程序的源代码调用 Send_Output 时，可以经该子程序声明的 out event 或 out event data port 发出相应事件或事件数据；端口声明本身不等于已经执行一次发送。

依据： 5.2(9)（第 68 页）；8.3.5(41)（第 135 页）。

5.2-07　按当前模式选择调用序列

带 in modes 的子程序调用序列仅在所列模式之一为当前模式时执行；当前模式不在集合时不参与执行；没有该子句时适用于全部模式。

依据： 5.2(11)（第 69 页）；12(11)（第 216 页）。

5.2-08 调用序列的默认执行次序

未由相应附录规定替代顺序时，子程序调用按调用声明先后顺序执行；附录明确给出替代顺序时采用该顺序。

依据： 5.2(14)–(15)（第 69 页）；5.2(23)（第 70 页）。

5.2-09 可重入代码的进程内共享

采用可重入源实现的分类器调用时，所属进程地址空间可以共享一份子程序二进制代码，不为每次调用另行复制。

依据： 5.2(16)（第 69 页）。

5.2-10 远程服务代码的所在进程

远程调用的实际子程序二进制代码位于服务线程所属进程的映像；调用方可装载代理，代理不改变实际服务代码的所属进程。

依据： 5.2(16)（第 69 页）。

5.2-11 显式子程序实例的地址空间

显式子程序子组件表示位于包含进程地址空间内的子程序实例。

依据： 5.2(17)（第 69 页）。

5.2-12 子程序访问连接的目标

requires/provides subprogram 或 subprogram group access 沿访问连接解析到实际子程序或子程序组；已明确指向另一线程服务的访问连接仍以该服务为目标。

依据： 5.2(17)（第 69 页）；8.4(2)–(3)（第 140 页）。

5.2-13 允许调用目标的候选集合

Allowed_Subprogram_Call 限定可服务该调用的子程序实例集合，候选可包含本地或远程实例；该集合本身不替代 Actual_Subprogram_Call 选择实际目标。对于通过该属性确定分类器调用目标的表示，未指定值时只允许本地调用。

依据： 5.2(18)（第 69 页）；A.1 Allowed_Subprogram_Call（第 244 页）。

5.2-14 实际调用目标的选择

通过调用属性解析分类器调用时，Actual_Subprogram_Call 确定实际服务实例；本地多处引用同一实例表示共享该代码，指向另一线程实例则使用该远程服务。此属性表示未给值时按本地调用解释，不覆盖已明确给出的访问连接目标。

依据： 5.2(18)（第 69 页）；A.1 Actual_Subprogram_Call（第 244 页）。

5.2-15 本地子程序由调用线程执行

未通过调用绑定改指其他线程服务的分类器调用、数据方法，以及访问连接指向非线程服务的子程序，由调用线程执行；目标代码位于其他 thread group、process 或 system 并不自行产生服务线程。

依据： 5.2(19)（第 69 页）；8.4(5)（第 140 页）。

5.2-16　远程子程序由提供线程执行

调用目标为另一线程的 provides subprogram access 时，由该提供线程执行；分类器调用经 Actual_Subprogram_Call 绑定到该入口时亦如此。

依据： 5.2(19)（第 69 页）；8.4(5)（第 140 页）。

5.2-17　设备服务的执行位置

调用由 device 提供的子程序服务时，相应服务在设备内部执行。

依据： 5.2(19)（第 69 页）。

5.2-18　处理器服务的执行位置

调用 processor 提供的子程序服务时，相应服务在该处理器内部的运行系统执行。

依据： 5.2(19)（第 69 页）。

5.2-19　本地同步调用的完成边界

本地子程序调用按同步方式完成；调用返回后，结果才供调用方后续动作使用。

依据： 5.2(20)（第 69 页）。

5.2-20　同步远程调用的阻塞和唤醒

Subprogram_Call_Type 为 Synchronous（默认）时，远程调用者进入 awaiting return 等待服务结果；结果返回后进入 ready，再参与调度。

依据： 5.2(20)（第 69–70 页）；5.4.3(49)（第 84 页）；A.2 Subprogram_Call_Type（第 252 页）。

5.2-21　半同步远程调用的结果依赖

实现支持且选用 SemiSynchronous 时，调用方可继续执行并发起其他远程调用；但后续动作需要某次尚未返回的结果时，必须等待该结果。实现允许仅支持同步调用。

依据： 5.2(20)、(25)（第 69–70 页）；A.2 Subprogram_Call_Type（第 252 页）。

5.2-22　服务线程的调用互斥

一个服务线程同时至多执行一个远程调用；互斥覆盖其全部 provides subprogram access 特征，不因请求不同入口而在同一线程并行服务。

依据： 5.2(21)（第 70 页）；8.4(9)（第 140 页）。

5.3　子程序组

5.3-01　子程序组访问的成员解析

requires subprogram group access 解析到提供访问的特征或具体子程序组子组件后，对组内子程序的连接和调用均指向该组中的相应成员，不因每个使用者访问该组而重新产生一个无关库实例。

依据： 5.3(3)–(4)（第 73 页）。

5.3-02　子程序组内的可选隐式实例

实现支持这种简化建模时，子程序组的 provides subprogram access 可以不连接显式子程序子组件，而作为组实例化时隐式声明并实例化的子程序解释；没有采用该许可时，不能据此假定所有未连接成员都有隐式实现。

依据： 5.3(6)（第 74 页）。

5.4　线程

5.4-01　输入冻结发生的时刻

线程默认在 dispatch 时冻结输入；对线程、处理器或设备指定其他 Input_Time 时，按指定时刻取新输入。event/event data 冻结项的选取同时服从适用出队协议。

依据： 5.4(3)（第 74 页）；8.3.2(15)、(18)（第 131 页）；8.3.2(17) Dispatch（第 131 页）；A.4 Input_Time（第 261 页）。

5.4-02　已冻结输入的稳定性

一次输入冻结后，后来到达的值不改变本轮源程序可见输入；直到下一规定冻结时刻或显式 Receive_Input 才更新。

依据： 5.4(3)（第 74 页）；8.3.2(15)、(19)（第 131 页）；9.2.4(30)（第 164 页）。

5.4-03　分派完成事件

线程一次正常执行（包括成功恢复）完成时，运行时通过已连接的 Complete out event port 发出完成事件；未连接该端口时，不因此产生向外传播的 Complete 连接事件。

依据： 5.4(5)、(11)（第 74、78 页）；5.4(11)（第 78 页）；8.3.4(37)（第 134 页）。

5.4-04　不可恢复结束的保留事件通知

线程在不可恢复错误条件下结束时，5.4(5) 规定经保留 Abort 和 Stop 端口交付事件；当模型显式声明并连接这些 out event 端口时，连接的接收端观察到相应通知。该结束通知与 Error 事件数据端口报告错误分别解释。

依据： 5.4(5)、(L3)、(C1)（第 74–75 页）。

5.4-05　不可恢复错误的端口传播

检测到线程不可恢复错误且 Error out event data port 已连接时，运行时经该端口传播错误事件数据；未连接时不通过该端口传播。

依据： 5.4(12)（第 78 页）；5.4.4(61)–(62)（第 86 页）。

5.4-06　线程内部子程序的外部可达性

线程中的子程序或专用子程序库可由该线程调用；只有通过相应 provides subprogram access 或 provides subprogram group access 暴露时，其他线程才通过这些接口调用所提供的服务。

依据： 5.4(13)（第 78 页）。

5.4-07　线程私有数据的访问边界

线程内部 data 由该线程独占访问；通过 provides data access 向外提供时，外部访问仍指向同一数据实例。

依据：5.4(14)（第 78 页）。

5.4.1　线程状态与动作

5.4.1-01　进程装载完成触发线程初始化

所属进程装载完成、发出 loaded(process) 后，halted 线程才开始执行已指定的初始化动作；此前不执行线程初始化。

依据：5.4.1(16)（第 78 页）；5.4.1(24)（第 79 页）。

5.4.1-02　初始化后的初始活动状态

线程初始化完成后，若属于初始活动模式则进入 awaiting dispatch，否则进入 awaiting mode；awaiting mode 中的线程在重新激活前不能接受普通分派。

依据：5.4.1(16)（第 78 页）；5.4.1 Figure 5（第 81 页）。

5.4.1-03　线程的初始状态

线程自动机的初始状态为 halted。

依据：5.4.1(16)（第 78 页）；5.4.1 Figure 5（第 81 页）。

5.4.1-04　普通停用路径进入等待模式

按普通停用路径处理且旧模式活动、新模式不活动的线程，执行已指定的 Deactivate_Entrypoint 后进入 awaiting mode；停用代码可按应用定义保存或重置状态。Background 及 zombie 线程采用其专门协议。

依据：5.4.1(18)（第 78–79 页）；5.4.1(28b)（第 80 页）；A.6 Deactivate_Entrypoint 系列（第 268 页）。

5.4.1-05　普通激活路径进入等待分派

按普通激活路径处理且旧模式不活动、新模式活动的线程，执行已指定的 Activate_Entrypoint 后进入 awaiting dispatch；激活代码可恢复或重置应用状态并设置新模式输出初值。Background 续执行及进程重新装载的替代路径仍采用其专门规则。

依据：5.4.1(18c)（第 79 页）；5.4.1(28b)（第 80 页）；A.6 Activate_Entrypoint、Activate_Entrypoint_Source_Text（第 266–267 页）。

5.4.1-06　分派后交给调度器

处于 awaiting dispatch 的活动线程被分派后进入计算动作并交给调度器；分派本身不等于已取得处理器开始运行。

依据：5.4.1(19)（第 79 页）；5.4.3(48a)（第 84 页）。

5.4.1-07　正常计算完成后等待新分派

本次计算成功完成后，线程返回 awaiting dispatch；没有新分派请求时不继续执行下一次计算入口。

依据： 5.4.1(19)（第 79 页）；5.4.3 Figure 6（第 85 页）。

5.4.1-08　计算期间不启动第二次并行分派

线程仍处于 compute 状态时，到达的事件或 event data 分派请求按相应端口队列机制处理，不同时启动该线程的第二个计算实例；队列满时采用所配置溢出协议。

依据： 5.4.1(20)（第 79 页）；8.3.3(30)–(31)（第 133 页）。

5.4.1-09　halted 线程不参与正常执行

线程处于 halted 时，不接受未来普通分派，也不参与正常模式切换；重新初始化后才重新进入相应生命周期。

依据： 5.4.1(21)、(28a)（第 79 页）。

5.4.1-10　线程受控终止的最终化路径

所属进程、虚拟处理器、处理器或系统的 stop 请求允许线程完成当前分派，再执行已指定的 Finalize_Entrypoint、相应 Source_Text 或 Call_Sequence 并进入 halted；不在当前活动模式的线程也可按该路径终止。

依据： 5.4.1(22)、(28e)（第 79–80 页）；5.4.1 Figure 5（第 81 页）；13.7(6)（第 231 页）；A.6 Finalize_Entrypoint 系列（第 269 页）。

5.4.1-11　中止请求立即停止线程

所属进程、虚拟处理器、处理器或系统的 abort 请求使线程立即停止当前执行并进入 halted，不等待当前分派正常完成。

依据： 5.4.1(22)（第 79 页）；5.4.1 Figure 5（第 81 页）。

5.4.1-12　线程终止释放锁定资源

线程通过终止或中止路径进入 halted 时，须释放此前通过 Get_Resource 锁定的资源；未释放资源属于错误状态。

依据： 5.4.1(22)（第 79 页）；5.4.1 Figure 5（第 81 页）；5.4.4 Figure 7（第 86 页）。

5.4.1-13　动态创建与初始化的许可

实现可以动态创建和初始化线程，包括在模式激活或停用时进行；该许可不取消线程开始初始化前所属进程须已装载的条件。

依据： 5.4.1(24)（第 79 页）。

5.4.1-14　初始化入口建立应用初值

线程初始化执行 Initialize_Entrypoint、相应 Source_Text 或 Call_Sequence 所指定的代码；该代码设置的应用状态及 out、in out 端口值作为后续执行的初始状态。

依据： 5.4.1(28a)（第 79 页）；A.6 Initialize_Entrypoint 系列（第 269–270 页）。

5.4.1-15　远程请求选择对应计算入口

远程子程序调用把服务线程分派到对应 provides subprogram access 的计算入口。不同访问特征分别标识各自入口。

依据： 5.4.1(28c)（第 80 页）；8.4(8)（第 140 页）。

5.4.1-16　事件特征入口覆盖线程入口

event 或 event data 触发分派时，有特征专用计算入口则执行该入口，否则执行线程级计算入口；不把两者都执行一遍。

依据： 5.4.1(28c)（第 80 页）；5.4.2(44)（第 83 页）；8.3.1(6)（第 130 页）；A.6 Compute_Entrypoint 系列（第 267–268 页）。

5.4.1-17　恢复阶段调用指定实现

线程需要恢复活动且已指定 Recover_Entrypoint、相应 Source_Text 或 Call_Sequence 时，执行相应恢复代码。

依据： 5.4.1(28d)（第 80 页）；A.6 Recover_Entrypoint 系列（第 270 页）。

5.4.1-18　缺失入口时不调用动作代码

动作既未通过适用入口属性指定代码，也未由附录指定实现时，不调用该动作入口；不自动借用其他阶段的入口。

依据： 5.4.1(29)（第 80 页）。

5.4.1-19　缺失计算入口时仍冻结输入

计算动作未指定任何适用入口或附录实现时，默认仍冻结特征，随后不执行进一步应用计算。

依据： 5.4.1(29)（第 80 页）。

5.4.1-20　初始化含恢复的累计经过时间

图 5 中初始化完成时，从初始化动作分派起累计的经过时间不得超过 Initialize_Deadline 与 Recover_Deadline 之和；发生恢复不重新开始这一累计计时。

依据： 5.4.1 Figure 5（第 81 页）；5.4.4(60)（第 86 页）。

5.4.1-21　初始化期限总和受超周期限制

图 5 要求 Initialize_Deadline + Recover_Deadline ≤ Hyper(Mode)；即使实际初始化提前完成，也须满足这一预算总和约束。

依据： 5.4.1 Figure 5（第 81 页）。

5.4.1-22　激活含恢复的累计经过时间

图 5 中线程激活动作完成时，从本次激活动作分派起的累计经过时间不得超过 Activate_Deadline 与 Recover_Deadline 之和。

依据： 5.4.1 Figure 5（第 81 页）；5.4.4(60)（第 86 页）。

5.4.1-23 停用含恢复的累计经过时间

图 5 中线程停用动作完成时，从本次停用动作分派起的累计经过时间不得超过 Deactivate_Deadline 与 Recover_Deadline 之和。

依据： 5.4.1 Figure 5（第 81 页）；5.4.4(60)（第 86 页）。

5.4.1-24 计算和恢复的累计经过时间上界

计算分派到再次等待分派的累计经过时间不得超过 Compute_Deadline + Recover_Deadline；恢复期间再出错不能在该总预算之外追加新的恢复期限。

依据： 5.4.1 Figure 5（第 81 页）；5.4.4(60)、Figure 7（第 86 页）。

5.4.2 线程分派

5.4.2-01 分派使能条件的触发

线程处于等待分派状态时，Enabled 由假变真触发分派并进入计算；Enabled 及 Wait_For_Dispatch 按当前分派协议解释。

依据： 5.4.2(30)、(35)–(36)（第 81–82 页）。

5.4.2-02 限定可触发分派的特征

线程分派协议使用特征触发时，Dispatch_Trigger 限定参与判断的特征集合；设备的该属性限定可触发分派的 event 或 event data 端口。未列入集合的输入不独立触发所指定分派。

依据： 5.4.2(37)（第 82 页）；A.2 Dispatch_Trigger（第 249 页）；6.6(13)（第 115 页）。

5.4.2-03 附录替代核心分派条件

Dispatch_Trigger_Annex=true 时，由相应附录给出 Enabled 条件，以该条件替代核心语言默认触发条件。

依据： 5.4.2(38)（第 82 页）；A.2 Dispatch_Trigger_Annex（第 249 页）。

5.4.2-04 Periodic 周期分派

Periodic 线程按 Period 指定的间隔获得周期分派；每次分派之后仍须通过调度才能取得处理器。

依据： 5.4.2(39)（第 82 页）；A.3 Period（第 256 页）。

5.4.2-05 Periodic 首次分派偏移

Periodic 线程设置 Dispatch_Offset 时，首次分派相对于其周期基准按指定时钟偏移量调整。

依据： 5.4.2(40)（第 82 页）；A.3 Dispatch_Offset（第 255 页）。

5.4.2-06 Aperiodic 分派

Aperiodic 线程等待分派时，只要参与触发的事件、事件数据或服务调用特征被激活，就满足分派条件；本协议不额外施加 Sporadic 所具有的 Period 最小分派间隔。

依据： 5.4.2(41)、(46)（第 82–83 页）；A.8 Supported_Dispatch_Protocols（第 275 页）。

5.4.2-07 Sporadic 分派

Sporadic 线程须同时满足存在激活的触发特征、距上次分派已达到 Period 两个条件才可再次分派；提前到达的触发不能缩短该最小间隔。

依据： 5.4.2(42)、(46)（第 82–83 页）；A.8 Supported_Dispatch_Protocols（第 275 页）。

5.4.2-08 Timed 特征触发的计算分支

Timed 线程等待分派时，参与触发的特征被激活会触发分派，并执行相应计算入口。

依据： 5.4.2(43)（第 82–83 页）。

5.4.2-09 Timed 超时触发的恢复分支

Timed 线程自上次分派达到 Period 仍未被特征触发时，发生超时分派；该超时分支按正文调用 Recover_Entrypoint。

依据： 5.4.2(43)（第 82–83 页）。

5.4.2-10 Hybrid 的两类触发来源

Hybrid 线程既可由周期条件触发，也可由参与分派的事件、event data 或调用特征触发；周期触发不以事件到达为前提，事件触发不须等待下一周期。入口选择仍按对应触发特征或线程入口。

依据： 5.4.2(44)、(46)（第 83 页）。

5.4.2-11 Background 首次激活即可分派

Background 线程初始化完成且首次处于活动模式时即可分派，不等待周期计时或输入事件；后续模式恢复按 Background 续执行路径处理。

依据： 5.4.2(45)（第 83 页）；5.4.3(50)（第 84 页）。

5.4.3 线程调度与执行

5.4.3-01 计算或恢复动作首先就绪

线程开始计算或恢复动作时首先进入 ready，而非直接假定已经在处理器运行。

依据： 5.4.3(47)–(48a)（第 84 页）。

5.4.3-02 调度器从就绪集合选择运行者

调度器需要选择下一个运行线程时，按所选调度协议从 ready 线程中选择，使被选线程进入 running。

依据： 5.4.3(48b)（第 84 页）。

5.4.3-03 无就绪候选时处理器等待

需要选择下一个运行线程而 ready 集合为空时，处理器保持空闲直至出现就绪线程；已有线程正在 running 时，ready 集合为空本身不停止该线程。

依据： 5.4.3(48b)（第 84 页）。

5.4.3-04　抢占使执行返回就绪

运行中的线程被调度器抢占时返回 ready，并保留尚未完成的本次执行；再次被选中后继续执行。

依据：5.4.3(48c)（第 84 页）；5.4.3 Figure 6（第 85 页）。

5.4.3-05　资源争用使线程阻塞

运行线程请求受保护共享资源而资源不可用时，停止执行并从 running 进入 awaiting resource。

依据：5.4.3(48d)（第 84 页）；5.4.3 Figure 6（第 85 页）。

5.4.3-06　资源释放使等待线程重新就绪

资源释放解除阻塞后，awaiting resource 线程返回 ready，仍须被调度器选中才能运行；资源可用不等于立即获得处理器。

依据：5.4.3(48e)（第 84 页）；5.4.3 Figure 6（第 85 页）。

5.4.3-07　Background 停用前离开临界区

Background 线程退出活动模式时暂停并进入 awaiting mode entry；若仍在临界区，须先退出临界区、释放全部资源后才暂停。

依据：5.4.3(50)（第 84 页）；5.4.3 Figure 6（第 85 页）。

5.4.3-08　Background 激活后续执行

暂停的 Background 线程重新进入活动模式时进入 ready，并从保留的执行位置继续；不从头启动一次新的后台计算。

依据：5.4.3(50)（第 84 页）；5.4.3 Figure 6（第 85 页）。

5.4.3-09　执行时钟只累计运行时间

动作的实际执行时间 c 只在线程 running 时累计；ready、资源等待、远程结果等待及模式重新进入等待均不累计 c。

依据：5.4.3(51)（第 84 页）；5.4.3 Figure 6（第 85 页）。

5.4.3-10　经过时间包含阻塞与排队

从动作分派起的经过时间 t 随真实时间累计；排队、抢占、资源阻塞及远程等待均不能从期限计量中扣除。

依据：5.4.3(51)（第 84 页）；5.4.3 Figure 6（第 85 页）。

5.4.3-11　专用执行时间覆盖线程预算

计算采用触发特征或入口专用 Execution_Time；未给专用值时使用线程相应预算。特征专用 Compute_Execution_Time 受线程级范围约束，远程调用特征的值约束服务线程。

依据：5.4.3(52)（第 84–85 页）；8.3.1(6)（第 130 页）；A.3 Compute_Execution_Time（第 253–254 页）。

5.4.3-12　专用期限覆盖线程期限

动作采用专用 Deadline，未给专用值时使用线程相应期限。触发特征的 Compute_Deadline 受线程级计算期限约束；远程调用特征的计算期限约束服务线程。

依据： 5.4.3(52)（第 84–85 页）；8.3.1(6)（第 130 页）；A.3 Compute_Deadline（第 253 页）。

5.4.3-13　正常动作的执行时间范围

无运行时错误的动作完成时，累计执行时间须位于对应 Execution_Time 的下、上界之间，执行中不得超过上界；各阶段分别采用相应 Initialize、Activate、Compute、Deactivate、Finalize 或 Recover 预算。

依据： 5.4.3(52)（第 84–85 页）；5.4.3 Figure 6（第 85 页）；A.3 各入口 Execution_Time 定义（第 253–257 页）。

5.4.3-14　正常动作的经过时间期限

正常动作从其计时起点到完成的经过时间不得超过对应 Deadline，等待调度或资源不暂停期限。Initialize、Activate、Compute、Deactivate、Finalize 和 Recover 分别使用该阶段的期限；涉及恢复的累计期限另按动作与恢复期限之和约束。

依据： 5.4.3(52)（第 84–85 页）；5.4.3 Figure 6（第 85 页）；A.3 各入口 Deadline 定义（第 253–256 页）。

5.4.3-15　正常完成前释放资源

图 6 的正常完成转移只在线程已释放所持共享资源时发生；完成计算不能留下仍被该线程锁定的资源。

依据： 5.4.3 Figure 6（第 85 页）。

5.4.3-16　线程切换不计入计算执行预算

在线程 running 与其他调度状态之间切换所耗的 Thread_Swap_Execution_Time 不计入该线程 Compute_Execution_Time；计算预算不含这一切换开销。

依据： 5.4.3(53)（第 85 页）。

5.4.3-17　时间约束与运行时检测的区别

违反已声明执行时间或期限约束属于异常执行；运行时可以在超限发生时强制检查并报告，也可以在事后检测并报告。仅有时间约束并不意味着调度器必须在边界瞬间中断线程或产生某个固定错误码。

依据： 5.4.3(54)（第 85 页）；6.1(21)（第 101 页）。

5.4.4　线程故障处理

5.4.4-01　应用处理优先于线程恢复

线程可恢复错误先允许由应用源码中的处理机制解决；应用已处理时不额外进入线程级恢复，未处理时受影响线程获得执行自身恢复入口的机会。实际恢复调用仍以指定了适用入口或附录实现为前提。

依据： 5.4.4(56)–(58)（第 85–86 页）；5.4.1(29)（第 80 页）。

5.4.4-02　成功恢复终结当前计算分派

计算分派的恢复入口成功完成后，本次分派视为完成，线程等待下一分派；不返回出错指令点继续本次计算。

依据：5.4.4(58)（第 85–86 页）；5.4.4 Figure 7（第 86 页）。

5.4.4-03　成功恢复释放持有资源

恢复完成时，须释放该线程持有的共享资源，使之后的执行可重新按资源协议访问。

依据：5.4.4(58)（第 85–86 页）；5.4.4 Figure 7（第 86 页）。

5.4.4-04　远程调用错误波及调用双方

远程子程序执行中发生线程可恢复错误时，服务该调用的线程和发起该调用的线程都获得恢复机会；是否实际调用恢复代码仍取决于相应线程是否指定恢复入口。

依据：5.4.4(59)（第 86 页）；5.4.1(29)（第 80 页）。

5.4.4-05　进入恢复时执行计时归零

动作进入恢复时，恢复实际执行时间 c 从零开始累计，不沿用出错前动作的执行时间作为恢复计数。

依据：5.4.4 Figure 7、(60)（第 86 页）。

5.4.4-06　进入恢复不重置累计经过时间

动作进入恢复时，不清零从原动作分派起累计的经过时间 t；恢复期间继续累计该时钟。

依据：5.4.4 Figure 7、(60)（第 86 页）。

5.4.4-07　计算和恢复的累计执行时间上界

本次计算与恢复的累计处理器执行时间不得超过相应两项 Execution_Time 上界之和；恢复期间再出错不追加另一份恢复执行预算。

依据：5.4.4(60)（第 86 页）。

5.4.4-08　错误识别码的确定来源

运行时检测到的错误使用实现规定的详细识别编码；应用主动传播错误时，可使用应用选择的编码。

依据：5.4.4(61)–(63)（第 86 页）。

5.4.4-09　不可恢复错误终止且不再恢复

线程发生不可恢复错误时，立即终止当前执行并进入 halted，不再调用 Recover_Entrypoint。

依据：5.4.4(62)、Figure 7（第 86 页）；5.4.1 Figure 5（第 81 页）。

5.4.4-10　错误报告是否携带数据

错误报告只需直接分派接收线程或触发模式转换时，可以发送 event；需要接收方区分错误细节时，以 event data 携带相应数据。

依据：5.4.4(63)（第 86 页）。

5.4.4-11　普通入口的时间故障可恢复

初始化、计算、激活或停用入口的时间故障按线程可恢复错误处理；该分类不自动扩展到正文未列的 finalize 时间故障。

依据： 5.4.4(64)（第 87 页）。

5.4.4-12　恢复期间的错误不可递归恢复

恢复入口时间故障或恢复期间发生的任何错误，均按线程不可恢复错误处理，不再递归执行一轮恢复。

依据： 5.4.4(64)–(65)（第 87 页）。

5.4.4-13　入口主动报告指定错误

线程入口可用 Raise_Error 按给定错误类型标识主动报告错误；恢复入口报告的错误必须按不可恢复类别处理。

依据： 5.4.4(66)（第 87 页）；5.4.8(84)（第 89 页）；A.9(15)（第 282 页）。

5.4.5　线程内部模式

5.4.5-01　线程内部当前模式的确定

线程内部当前模式在确定线程输入时确定；它可由应用代码隐式决定，或由显式模式转换响应输入、self 事件及子程序调用产生的事件。

依据： 5.4.5(69)–(70)（第 87 页）。

5.4.6　同步执行语义

5.4.6-01　局部动作时钟与域时钟分离

同一同步域共享参考时间 ST，各线程仍保有各自动作的局部 c、t；重置线程局部时钟不把运行中的域参考时间归零。

依据： 5.4.6(71)（第 87 页）。

5.4.6-02　周期集合的超周期计算

同一同步域一组周期线程的 hyperperiod 为其周期在统一时间单位下的最小公倍数，表示这些周期分派再次对齐的间隔。

依据： 5.4.6(72)（第 87 页）；13.4(3)（第 227 页）。

5.4.6-03　同步周期分派的逻辑对齐

同一同步域中，在超周期边界，周期线程的控制和数据交换顺序须等价于同时分派；实现中的实际执行先后不改变这种逻辑对齐。

依据： 5.4.6(73)（第 87 页）；13.4(2)–(3)（第 227 页）。

5.4.7 异步执行语义

5.4.7-01 不同域的参考时钟分别解释

不同同步域可使用不同参考时间；没有适用协调协议时，不能把不同域时间戳直接解释为同一时钟读数。

依据： 5.4.7(75)–(76)（第 87–88 页）。

5.4.7-02 线程采用执行处理器的时间源

线程执行采用实际执行处理器使用的参考时间，不能为同一执行随意替换为其他同步域时钟。

依据： 5.4.7(77)（第 88 页）；A.3 Reference_Time（第 258 页）。

5.4.7-03 通信涉及各参与方时间源

通信时间的解释同时涉及源、目的以及参与定时通信的平台组件所用参考时间，不能只用发送线程时间源代替整条路径。

依据： 5.4.7(77)（第 88 页）；A.3 Reference_Time（第 258 页）。

5.4.8 线程运行时支持

5.4.8-01 Await_Dispatch 确定恢复执行点

本次分派完成时调用 Await_Dispatch，执行在该调用处挂起，下一次分派从该点恢复；该调用本身不表示立即执行一次新分派。

依据： 5.4.8(82)（第 88 页）；A.9(13)（第 281–282 页）。

5.4.8-02 Await_Dispatch 的状态配置

Await_Dispatch 的 State 配置分派触发条件，内部表示由实现决定；实现可以从原模型提供默认 State 并检查其有效性。

依据： 5.4.8(82)（第 88 页）；A.9(13)（第 281 页）。

5.4.8-03 Await_Dispatch 返回调用状态

Await_Dispatch 返回的 Error_Code 表明本次服务调用是否成功。

依据： 5.4.8(82)（第 88 页）；A.9(13)（第 281 页）。

5.4.8-04 查询实际分派原因

Dispatch_Status 返回导致当前分派的条件信息，例如相应触发特征或定时事件；返回信息描述实际已发生的分派原因，不等于声明中所有可能触发源的列表。

依据： 5.4.8(83)（第 88–89 页）；A.9(14)（第 282 页）。

5.4.8-05　查询导致恢复的错误类型

恢复入口可调用 Get_Error_Code 取得导致本次恢复的错误类型标识；查询结果描述该次错误。

依据： 5.4.8(85)（第 89 页）；A.9(16)（第 282 页）。

5.4.8-06　子程序错误归实际执行线程

子程序没有自己的线程 Error 端口，子程序内 Raise_Error 作用于实际执行它的线程；远程子程序的错误报告给服务线程，而不是自动改为调用方的 Error 端口。

依据： 5.4.8(86)（第 89 页）；A.9(17)（第 282 页）。

5.4.8-07　按调用标识等待远程结果

Await_Result 以 Call_ID 指定要等待的半同步调用，挂起应用直至该调用的结果可用。

依据： 5.4.8(87)（第 89 页）；A.9(18)（第 282 页）。

5.4.8-08　非抢占调度关闭抢占转换

Preemptive_Scheduler=false 时，调度器不通过抢占将 running 线程移回 ready；完成、阻塞或故障仍可使线程离开 running。该属性未指定时默认 true。

依据： 5.4.8(88)（第 89 页）；A.1 Preemptive_Scheduler（第 248 页）。

5.5　线程组

5.5-01　线程组不产生独立执行单元

线程组用于组织内部线程，不自身产生可调度执行单元。

依据： 5.5(2)（第 90 页）。

5.5-02　线程组不建立地址空间

线程组不建立独立虚拟地址空间；组内子程序和子程序组代码位于所属进程地址空间。

依据： 5.5(2)、(4)（第 90–91 页）。

5.5-03　线程组对外提供子程序访问

线程组内子程序可由内部线程调用；向组外提供调用时，使用相应 provides subprogram 或 subprogram group access，外部调用仍解析到该组中的成员。

依据： 5.5(4)（第 91 页）。

5.5-04　线程组内部数据的共享路径

线程组内 data 通过组内访问连接供线程共享，向组外共享时通过线程组 provides data access 暴露；接口不复制该数据实例。

依据： 5.5(5)（第 91 页）。

5.6 进程

5.6-01 进程不隐式创建主线程

进程的执行线程来自显式 thread 子组件声明，包括经线程组包含的线程；process 本身不额外创建隐式主线程。

依据： 5.6(1)（第 92 页）。

5.6-02 每进程拥有独立地址空间

每个进程具有自己的虚拟地址空间；该进程内部组件的代码和数据经该空间访问，所含线程在该空间内执行。

依据： 5.6(1)–(4)（第 92–94 页）。

5.6-03 地址空间的运行时保护

Runtime_Protection 有效值为 true 时执行进程地址空间边界保护，false 时可关闭；未指定时默认 true。

依据： 5.6(2)（第 93 页）；A.2 Runtime_Protection（第 252 页）。

5.6-04 装载经过时间不超过期限

进程从开始装载到完成装载的经过时间不得超过 Load_Deadline，违反这一要求视为错误。

依据： 5.6(7)（第 94 页）；A.3 Load_Deadline（第 256 页）。

5.6-05 进程装载执行时间范围

无运行时错误时，装载进程映像所需执行时间位于 Load_Time 规定范围；预装载允许零耗时的情形另依相应装载许可。

依据： 5.6(7)–(8)（第 94 页）；A.3 Load_Time（第 256 页）。

5.6-06 进程装载等待所绑定平台启动

绑定到 processor 的进程在该 processor 启动后可开始装载；绑定到 virtual processor 时，须等待对应 virtual processor 启动。

依据： 5.6(8)（第 94 页）；5.6 Figure 8（第 94–95 页）。

5.6-07 预装载映像允许零装载耗时

二进制映像已预装入 ROM、PROM 或 EPROM 时，系统启动中的相应装载操作允许耗时为零。

依据： 5.6(8)（第 94 页）。

5.6-08 进程受控停止使全部线程最终化

Stop_Process 或所绑定平台的停止使进程受控关闭；所含线程无论正在执行、等待分派还是不属于当前模式，都获得执行已指定 finalize 入口后 halt 的机会。

依据： 5.6(9)（第 94 页）；13.7(6)（第 231 页）；A.9(24)（第 283 页）。

5.6-09　进程中止立即终止全部线程

Abort_Process 立即中止该进程所含全部线程并释放其锁定资源，不等待当前分派或受控最终化流程完成。

依据： 5.6(10)（第 94 页）；13.7(7)（第 231 页）；A.9(25)（第 283 页）。

5.6-10　装载完成进入进程启动状态

图 8 中装载完成后，进程由 loading 进入 starting，并清零该启动动作的 t 和 c。

依据： 5.6 Figure 8（第 94–95 页）。

5.6-11　进程启动完成的期限守卫

图 8 中进程从 starting 完成启动时，须满足 t≤Startup_Deadline，随后发出 loaded(process) 并进入 loaded。

依据： 5.6 Figure 8（第 94–95 页）；A.3 Startup_Deadline（第 257 页）。

5.6-12　进程启动动作的执行预算

图 8 在 starting 中令 c 与 t 均以单位速率增长，并在完成启动时断言 t≤max(Startup_Execution_Time)；该预算描述进程自身启动，线程初始化由自己的 initialize 预算计量。

依据： 5.6 Figure 8（第 94–95 页）；A.3 Startup_Execution_Time（第 257 页）。

5.6-13　装载错误返回未装载状态

图 8 中进程装载检测到错误时，从 loading 返回 unloaded。

依据： 5.6 Figure 8（第 94–95 页）。

5.6-14　进程中止返回未装载状态

图 8 中 loading、starting 或 loaded 状态收到进程或所绑定平台的 abort 时，返回 unloaded。

依据： 5.6 Figure 8（第 94–95 页）。

5.6-15　进程停止请求进入 stopping

图 8 中 loaded 进程收到自身或所绑定平台的 stop 时，进入 stopping 状态。

依据： 5.6 Figure 8（第 94–95 页）。

5.6-16　进程停止完成后进入 unloaded

图 8 中 stopping 进程收到相应 stopped 完成信号后回到 unloaded 状态。

依据： 5.6 Figure 8（第 94–95 页）。

5.6-17　动态内存和链接仍满足期限

实现允许在进程装载完成且线程开始执行后使用动态虚拟内存管理或动态库链接，但各线程的 deadline 约束仍须满足。

依据： 5.6(13)（第 95 页）；13.7(23)（第 232 页）。

第 6 章　执行平台组件

6-01　平台软件绑定到可达内存

处理器调度及其他平台服务由软件实现时，其代码和数据二进制映像绑定到执行处理器可访问的内存；虚拟处理器功能软件同样由 Actual_Memory_Binding 确定这类内存位置。

依据： 6(8)（第 96 页）；6.1(3)（第 99 页）；6.2(3)（第 103 页）；13.2(C10)（第 223 页）。

6-02　处理器属性包含平台软件资源需求

处理器属性所描述的资源需求包含实现处理器调度、执行等功能的软件开销；平台软件不因隐藏在处理器抽象内部而不计入该资源描述。

依据： 6(8)（第 96 页）。

6.1　处理器

6.1-01　处理器提供调度协议集合

Allowed_Dispatch_Protocol 确定处理器或虚拟处理器可执行的线程调度协议；未指定该列表时，线程可采用任意调度协议绑定并执行。该支持集合也用于虚拟处理器接纳其上绑定的线程或虚拟处理器。

依据： 6.1(2)（第 99 页）；6.2(2)（第 103 页）；A.1 Allowed_Dispatch_Protocol（第 247 页）。

6.1-02　线程代码和数据内存对执行处理器可达

线程执行的代码及所访问数据绑定的内存，必须对实际执行该线程的处理器可达。

依据： 6.1(4)（第 99 页）。

6.1-03　驱动所控制设备对执行处理器可达

处理器执行某设备的驱动软件时，该处理器必须能够访问对应设备。

依据： 6.1(5)（第 99 页）。

6.1-04　处理器初始停止

处理器自动机的初始状态为 stopped。

依据： 6.1(9)（第 99 页）。

6.1-05　启动请求使处理器进入 starting

处于 stopped 的处理器收到 start(system) 或 start(processor) 时进入 starting。

依据： 6.1(9)（第 99 页）；图 9（第 100 页）。

6.1-06　处理器启动初始化硬件

处理器处于 starting 时执行处理器硬件初始化。

依据： 6.1(9)（第 99 页）。

6.1-07 处理器启动装载调度软件

处理器进入 starting 时，将提供线程调度功能的处理器软件装载到内存。

依据： 6.1(9)（第 99 页）。

6.1-08 处理器启动初始化调度软件

处理器处于 starting 时，初始化提供线程调度功能的处理器软件。

依据： 6.1(9)（第 99 页）。

6.1-09 进程映像可并入平台统一映像

进程虚拟地址空间映像允许已经随包含处理器或虚拟处理器软件的统一装载映像一起加载；处理器启动不要求把这样的进程映像再次独立装载。

依据： 6.1(9)（第 99 页）。

6.1-10 处理器就绪后进入 operational

处理器及其管理虚拟处理器或进程的软件初始化完成后，经 started(processor) 转换进入 operational。

依据： 6.1(9)（第 99 页）。

6.1-11 下层执行实体在处理器初始化后启动

处理器进入 operational 并发出 started(processor) 之后，才开始所关联虚拟处理器、进程及随后线程的初始化。

依据： 6.1(9)（第 99 页）。

6.1-12 处理器中止先中止绑定线程

处理器 abort 首先使绑定到该处理器的线程中止。

依据： 6.1(11)（第 99 页）。

6.1-13 处理器随后中止下层虚拟处理器

处理器 abort 中止绑定线程之后，使包含于其中或绑定到它的虚拟处理器中止。

依据： 6.1(11)（第 99 页）；13.7(11)（第 231 页）。

6.1-14 处理器停止在超周期边界开始

停止处理器请求在绑定到该处理器或其虚拟处理器的周期线程下一超周期边界使处理器进入 stopping；同步组件选择可减少参与超周期计算的线程。

依据： 6.1(12)（第 99 页）；13.7(10)（第 231 页）。

6.1-15 处理器停止通知虚拟处理器

到达处理器停止边界时，相关虚拟处理器收到停止通知；Stop_Processor 对全部绑定虚拟处理器发起 Stop_Virtual_Processor。

依据： 6.1(13)（第 100 页）；13.7(10)（第 231 页）。

6.1-16　处理器停止通知进程

到达处理器停止边界时，含有绑定线程的相关进程收到停止通知；Stop_Processor 对全部绑定进程发起 Stop_Process。

依据： 6.1(13)（第 100 页）；13.7(10)（第 231 页）。

6.1-17　处理器有序停止允许当前分派完成

处理器在超周期边界发起有序停止时，绑定线程获准完成当前分派执行，再停止其所属进程。

依据： 6.1(13)（第 100 页）。

6.1-18　处理器有序停止允许线程最终化

处理器在超周期边界发起有序停止时，绑定线程获准执行所需 finalization；其所属进程在该最终化完成后停止。

依据： 6.1(13)（第 100 页）。

6.1-19　处理器启动复位局部时钟

处理器从 stopped 进入 starting 的启动转换执行 t←0。

依据： 图 9（第 100 页）。

6.1-20　处理中止返回 stopped

图 9 中 starting 或 operational 的处理器收到 abort(system) 或 abort(processor) 时返回 stopped。

依据： 图 9（第 100 页）。

6.1-21　处理器致命错误返回 stopped

图 9 中 starting 或 operational 的处理器发生 processor fatal error 时返回 stopped。

依据： 图 9（第 100 页）。

6.1-22　处理器停止完成返回 stopped

图 9 中处于 stopping 的处理器经 stopped(system) 或 stopped(processor) 完成停止并进入 stopped。

依据： 图 9（第 100 页）。

6.1-23　重复停止保持 stopped

图 9 中已经 stopped 的处理器再次收到 stop(processor) 时保持 stopped。

依据： 图 9（第 100 页）。

6.1-24　系统时刻零开始的处理器启动期限

处理器在系统时刻 ST=0 开始启动且局部时钟 t 同时复位时，启动完成所经过的时间不超过 Startup_Deadline。

依据： 图 9（第 100 页）。

6.1-25　处理器服务由线程绑定定位

应用调用处理器提供的子程序或子程序组服务时，调用由子程序分类器标识，具体服务处理器通过包含该调用的线程的实际处理器绑定隐式确定。

依据： 6.1(15)（第 100 页）。

6.1-26　协议能力不要求显式虚拟总线

processor、virtual processor、bus、virtual bus、device、memory 及 system 可以通过 Provided_Virtual_Bus_Class 声明所支持的协议；在允许声明相应虚拟总线子组件的类别中，也可以显式声明协议实例。仅有该属性即可表明协议能力，无须为了提供协议而必定建立虚拟总线子组件。

依据： 6.1(16)（第 100 页）；6.2(4)（第 103 页）；6.4(10)（第 107–108 页）；6.6(19)（第 115 页）；A.1 Provided_Virtual_Bus_Class（第 244–245 页）。

6.1-27　管理内部总线者隐式连接

处理器包含一个向外提供访问的总线子组件时，该处理器隐式连接到这条由其管理的总线；设备包含并向外提供内部总线时同样隐式连接到该总线。

依据： 6.1(17)（第 100 页）；6.6(18)（第 115 页）。

6.1-28　绑定专属执行时间优先

线程或子程序对实际绑定处理器已有 in binding 执行时间关联时，采用该处理器专属值，不再以 Reference_Processor 缩放替代该值。

依据： 6.1(18)（第 100 页）；13.1(8)（第 222 页）；A.3 Reference_Processor（第 258 页）。

6.1-29　参考执行时间按处理能力缩放

未给目标处理器专属执行时间且已有 Reference_Processor 时，参考执行时间乘以参考处理器容量与目标处理器容量之比，得到目标上的执行时间；范围的两个端点采用同一缩放因子。

依据： 6.1(18)（第 100 页）；13.1(8)（第 222 页）；A.3 Reference_Processor、Processor_Capacity（第 258 页）。

6.1-30　处理器容量是预算速度量

Processor_Capacity 以每时间单位的指令数描述处理器容量或速度，供处理能力预算分析使用。

依据： 6.1(18)（第 100 页）；A.3 Processor_Capacity（第 258 页）。

6.1-31　平台实现关联到系统模型

Implemented_As 把 processor、virtual processor、memory、bus、virtual bus、device 或 system 关联到描述其内部构造的 system implementation 或 abstract implementation；这种关联为平台抽象提供下层实现模型，本身不等同于声明一个普通包含子组件。采用 system implementation 进行实例替换或访问重映射时，另按第 14 章对应组件类别的规则处理。

依据： 14(5)–(6)（第 233 页）；6.1(19)（第 100 页）；6.2(9)（第 104 页）；6.3(5)（第 105–106 页）；6.4(12)（第 108 页）；6.5(12)（第 111 页）；6.6(20)（第 115 页）；A.7 Implemented_As（第 274 页）。

6.1-32 处理器启动期限可以不被监测

实现方法不必监测处理器 startup deadline，也不必把超限报告为错误；未报告超限不能证明模型的启动期限约束得到满足。

依据： 6.1(20)（第 101 页）。

6.1-33 可选 deadline 检查报告可恢复错误

实现选择强制检查线程 deadline 时，deadline 违反被报告为线程可恢复错误；核心标准不要求所有实现都开启此检查。

依据： 6.1(21)（第 101 页）。

6.1-34 可选计算预算检查报告可恢复错误

实现选择强制检查线程最大计算执行时间时，超过该上界被报告为线程可恢复错误；核心标准不要求所有实现都开启此检查。

依据： 6.1(21)（第 101 页）。

6.2 虚拟处理器

6.2-01 虚拟处理器的容器确定隐式绑定

虚拟处理器作为处理器或另一虚拟处理器的子组件时，隐式绑定到该容器；作为 system 子组件单独声明时，由 Actual_Processor_Binding 显式确定执行它的处理器或虚拟处理器。

依据： 6.2(1)（第 101 页）；6.2(5)（第 103 页）；A.1 Actual_Processor_Binding（第 242 页）。

6.2-02 虚拟处理器链确定物理执行位置

在完整绑定系统中，沿线程或虚拟处理器的执行绑定逐层追踪，最终到达实际物理处理器；该终点决定其硬件执行位置。

依据： 6.2(1)（第 101 页）。

6.2-03 虚拟处理器初始化遵循外到内次序

虚拟处理器在其绑定的处理器和外层虚拟处理器初始化完成后、其进程及线程初始化之前初始化。

依据： 6.2(8)（第 103 页）。

6.2-04 虚拟处理器最后停止

虚拟处理器在其内部线程、进程和虚拟处理器停止之后停止，不先于这些下层实体完成自己的停止。

依据： 6.2(8)（第 103 页）。

6.2-05 虚拟处理器启动进入 starting

图 10 中处于 stopped 的虚拟处理器收到 started(processor) 或 start(vprocessor) 后进入 starting。

依据： 图 10（第 104 页）。

6.2-06　虚拟处理器启动时钟归零

图 10 中虚拟处理器从 stopped 启动进入 starting 时执行 t←0。

依据：图 10（第 104 页）。

6.2-07　虚拟处理器完成启动进入 operational

图 10 中虚拟处理器经 started(vprocessor) 完成启动并进入 operational；该转换断言 t≤startup_deadline。

依据：图 10（第 104 页）。

6.2-08　虚拟处理器停止在超周期边界开始

图 10 中运行的虚拟处理器因 stop(vprocessor) 或 stop(processor)，在相应超周期边界进入 stopping；Stop_Virtual_Processor 也在下一超周期发起该转换。

依据：图 10（第 104 页）；13.7(8)（第 231 页）。

6.2-09　虚拟处理器完成停止返回 stopped

图 10 中 stopping 经 stopped(vprocessor) 或 stopped(processor) 标示的停止完成转换返回 stopped。

依据：图 10（第 104 页）。

6.2-10　虚拟处理器重复停止保持 stopped

图 10 中已经 stopped 的虚拟处理器再次收到 stop(vprocessor) 时保持 stopped。

依据：图 10（第 104 页）。

6.2-11　虚拟处理器中止返回 stopped

图 10 中处于 starting 或 operational 的虚拟处理器收到 abort(system)、abort(processor) 或 abort(vprocessor) 时返回 stopped。

依据：图 10（第 104 页）。

6.2-12　虚拟处理器致命错误返回 stopped

图 10 中虚拟处理器启动或运行时发生 Vprocessor fatal error，则返回 stopped。

依据：图 10（第 104 页）。

6.3　存储器

6.3-01　模式事件可沿平台层次下传

memory 或 bus 内部的端口连接可将模式事件触发沿包含层次向下传播。

依据：6.3(L4)（第 105 页）；6.4(13)（第 108 页）。

6.3-02　内存映像对所属进程线程开放

代码和数据映像加载到表示进程虚拟地址空间的内存后，供该进程中在相应处理器上执行的线程访问；该内存须通过包含关系或共享总线对处理器可达。

依据： 6.3(2)（第 105 页）。

6.3-03　二进制映像可以预装

二进制映像可在处理器启动期间加载，也可以在系统启动前已经预装于内存；内存中的映像不必都由运行时启动流程重新加载。

依据： 6.3(2)（第 105 页）。

6.3-04　内存可达性的结构路径

内存包含在处理器内，或内存与处理器通过共享总线相连时，可以建立处理器访问内存的结构路径；共享总线访问还受总线物理访问许可约束。

依据： 6.3(3)（第 105 页）；6.4(6)（第 107 页）；A.1 Allowed_Physical_Access（第 247 页）。

6.3-05　平台端口事件可触发模式转换

带模式的 memory、bus 及 virtual bus 可以用端口作为其模式转换触发器。

依据： 6.3(6)（第 106 页）；6.4(13)（第 108 页）；6.5(13)（第 111 页）。

6.4　总线

6.4-01　总线间可直接连接

一条总线通过 requires bus access 要求访问另一条总线时，形成总线之间的直接连接；两条总线可使用不同的总线分类器。

依据： 6.4(3)（第 106 页）；6.4(7)（第 107 页）。

6.4-02　跨处理器连接使用总线传输

连接两端的软件组件绑定到不同处理器时，其信息通过支持该连接类别的总线传输。

依据： 6.4(4)（第 106 页）。

6.4-03　总线支持的连接类别由属性决定

Allowed_Connection_Type 列出总线支持的连接类别；没有该列表时，任意连接类别都不受这项属性排除。

依据： 6.4(5)（第 107 页）；A.1 Allowed_Connection_Type（第 247 页）。

6.4-04　共享总线使设备可达

设备与处理器共享总线组件时，该设备对处理器可达。

依据： 6.4(3)（第 106 页）；6.4(6)–(7)（第 107 页）；6.6(8)（第 114 页）。

6.4-05　协议要求由提供方或协议实例满足

连接或虚拟总线的 Required_Virtual_Bus_Class 要求与绑定目标的 Provided_Virtual_Bus_Class 匹配；若协议被显式建模成虚拟总线实例，也可将要求该协议的连接或虚拟总线直接绑定到该实例。

依据： 6.4(10)（第 107–108 页）；6.5(8)（第 111 页）。

6.4-06　软件协议在通信处理器执行

虚拟总线代表的协议由软件实现时，该软件在连接到相应总线的处理器上执行，这些处理器上绑定的线程通过要求该协议的连接通信。协议也允许在总线硬件中实现。

依据： 6.4(11)（第 108 页）。

6.5　虚拟总线

6.5-01　虚拟总线绑定形成协议层次

虚拟总线绑定到另一虚拟总线时，形成表示分层协议服务的虚拟通道层次。

依据： 6.5(2)（第 109 页）。

6.5-02　虚拟总线包含产生隐式绑定

虚拟总线作为处理器、总线或另一虚拟总线的子组件时，隐式绑定到该容器。

依据： 6.5(4)（第 109 页）。

6.5-03　连接服务质量由绑定平台满足

连接或虚拟总线由 Required_Connection_Quality_Of_Service 声明的传输服务质量要求，须由实际绑定的总线、虚拟总线或提供协议的执行平台组件以 Provided_Connection_Quality_Of_Service 所描述的能力满足。

依据： 6.5(5)（第 109 页）；A.1 Provided_Connection_Quality_Of_Service 与 Required_Connection_Quality_Of_Service（第 245 页）。

6.6　设备

6.6-01　设备数据保存内部持久状态

设备内部的 data 子组件保存设备的持久状态，设备 Behavior Annex 引用该内部数据实例；该状态不通过 data access 向外提供。

依据： 6.6(11)（第 114 页）。

6.6-02　设备协议决定周期或事件驱动

设备 Dispatch_Protocol 决定其按周期轮询环境或采样应用输入，还是按事件产生、转发或响应交互；事件驱动设备不因该协议而自动增加周期性采样。

依据： 6.6(13)（第 115 页）。

6.6-03　设备端口速率由属性描述

Output_Rate 描述设备端口的输入或输出速率，速率取其有效属性值。

依据： 6.6(13)（第 115 页）。

6.6-04　驱动接口采用设备分派协议

设备接口由驱动提供时，设备 Dispatch_Protocol 描述该驱动的执行分派行为。

依据： 6.6(14)（第 115 页）。

6.6-05　设备代码量计入内核驱动代码

设备驱动作为操作系统内核一部分执行且以设备属性表征驱动资源时，设备的 Code_Size 表示该驱动的静态代码及只读数据大小。

依据： 6.6(15)（第 115 页）；A.5 Code_Size（第 265 页）。

6.6-06　设备数据量计入内核驱动数据

设备驱动作为操作系统内核一部分执行且以设备属性表征驱动资源时，设备的 Data_Size 表示该驱动的可读写数据大小。

依据： 6.6(15)（第 115 页）；A.5 Data_Size（第 265 页）。

6.6-07　设备栈大小计入内核驱动栈

设备驱动作为操作系统内核一部分执行且以设备属性表征驱动资源时，设备的 Stack_Size 表示该驱动的执行期间使用的最大栈大小。

依据： 6.6(15)（第 115 页）；A.5 Stack_Size（第 265 页）。

6.6-08　设备实际绑定确定驱动执行平台

device 的 Actual_Processor_Binding 确定设备驱动软件的执行平台；内核驱动绑定到包含该驱动的运行时系统所属处理器，驱动在独立于内核的地址空间执行时，可以以 virtual processor 作为绑定目标。

依据： 6.6(15)–(16)（第 115 页）；A.1 Actual_Processor_Binding（第 242 页）。

6.6-09　应用驱动由显式线程执行

设备驱动被建模为应用线程时，该 AADL 线程执行驱动逻辑，一侧为应用提供设备接口，另一侧访问由设备组件表示的设备寄存器。

依据： 6.6(17)（第 115 页）。

6.6-10　隐式驱动的执行主体由实现选择

使用属性隐式表示设备驱动时，实现可将其执行视为所连处理器运行时系统的一部分，也可将其视为隐式声明线程的执行。

依据： 6.6(22)（第 115 页）。

第 7 章　组合组件

7.1　系统

7.1-01　硬件系统标志

Is_Hardware 为真时，system 作为硬件系统解释，例如多核处理器、片上系统或设备；system 可包含软硬件组件，实际组成仍由其子组件及绑定决定。

依据： 7.1(5)（第 119 页）；A.7 Is_Hardware（第 274 页）。

第 8 章　特征

8.2　特征组

8.2-01　互补组成员按声明顺序配对

互补特征组按成员声明顺序逐对对应，嵌套成员组继续按该对应关系解释；存在类型扩展时，继承成员先于扩展新增成员参与配对。两个实际空组的成员配对为空。

依据： 8.2(L9)（第 125 页）；8.2(L10)（第 125 页）。

8.2-02　互补端口采用可通信方向

inverse of 所表示的互补特征组中，成对 port 成员按端口连接规则对应：输入端与支持输出的端对应，输出端与支持输入的端对应，in out 端仍具双向能力。

依据： 8.2(L11)（第 125 页）；8.2(5)（第 126 页）；9.2(L6)–(L8)（第 158 页）。

8.2-03　互补访问成员匹配提供与需求

inverse of 所表示的互补特征组中，access 成员按访问连接规则使 provides 与 requires 角色对应；data access 的实际读写流向仍受其 Access_Right 及连接方向约束。

依据： 8.2(L12)（第 126 页）；8.2(5)（第 126 页）；9.4(L5)–(L7)（第 172 页）。

8.3　端口

8.3-01　data 到达默认不触发分派

新的数据到达 data 端口默认不触发线程 dispatch；线程是否分派由其分派协议及所配置触发条件决定。

依据： 8.3(3)b（第 128 页）。

8.3.1　端口类别

8.3.1-01　端口映射到静态存储变量

data 和 event data 端口在源程序中映射为表示其数据缓冲区或队列的静态变量。

依据： 8.3.1(5)（第 130 页）。

8.3.1-02　端口变量的名称映射

未显式改变名称映射时，data 或 event data 端口的源程序变量与端口同名；指定 Source_Name 及关联 Source_Text 时按指定映射访问该变量。

依据： 8.3.1(5)（第 130 页）。

8.3.1-03　in out 端口保留两种连接角色

in out 端口同时作为 incoming 和 outgoing 接口；入连接的源组件与出连接的目标组件可相同，也可不同。

依据： 8.3.1(7)（第 130 页）。

8.3.1-04　in out data 写入覆盖输入值

in out data 端口的输入和输出映射到同一个端口变量；源程序写入输出值时覆盖该变量此前保存的输入值。

依据： 8.3.1(7)（第 130 页）。

8.3.1-05　事件数据输出允许独立变量

event data 端口实现允许为输出值使用独立于冻结输入队列的端口变量。

依据： 8.3.1(7)（第 130 页）。

8.3.1-06　data 输入仅保留最近样本

data 端口不积累历史消息队列；两次输入获取之间有多个新值到达时，接收方取得最近到达的值，较早值被覆盖。

依据： 8.3.1(12)（第 131 页）；8.3(3)b（第 128 页）；9.2.5(51)（第 166 页）。

8.3.1-07　data 没有新输入时重复旧值

已接收过数据的 data 端口在下次输入获取前没有新值到达时，再次提供此前值；可读到该值并不表示该值在本次 dispatch 是新鲜值。

依据： 8.3.1(12)（第 131 页）；9.2.5(52)（第 166 页）。

8.3.1-08　空事件数据队列仍可读最近值

event data 端口先前已接收过数据而当前可用队列为空时，最近的数据值仍可读取；是否有新项由可用项数判断。

依据： 8.3.1(13)（第 131 页）。

8.3.1-09　事件类 Get_Count 返回可用项数

对 event 或 event data 端口调用 Get_Count，返回按出队协议已提供给线程的项数；0 表示没有新项可用，不把所有仍在底层队列中的项一概计为本次可用项。

依据： 8.3.1(13)–(14)（第 131 页）；8.3.3(34)（第 133 页）；8.3.5(45)b（第 136 页）；A.9(7)b（第 280 页）。

8.3.2 输入与输出时刻

8.3.2-01 Start 偏移内冻结输入

端口有效 Input_Time 为 Start 时，自实际开始执行累计的执行时间落入所指定正偏移区间时冻结输入；偏移按执行时间计算，不按等待调度期间的经过时间计算。

依据： 8.3.2(17) Start（第 131 页）；A.4 Input_Time、IO_Time_Spec（第 261 页）。

8.3.2-02 NoIO 不提供端口新输入

端口有效 Input_Time 为 NoIO 时，不为源程序冻结和提供该端口的新输入；属性随模式选择为其他值后，按新有效值处理。

依据： 8.3.2(17) NoIO（第 131 页）。

8.3.2-03 Dynamic 允许执行中任意输入时刻

端口有效 Input_Time 为 Dynamic 时，输入可以在执行期间任意时刻发生，不要求相对 Start 或 Completion 的特定偏移。

依据： 8.3.2(17) Dynamic（第 131 页）；A.4 IO_Reference_Time（第 261 页）。

8.3.2-04 一轮分派可以多次冻结输入

Input_Time 列出多个输入时刻时，一次 dispatch 执行中按这些时刻多次冻结输入，各次冻结更新当次提供给源程序的内容。

依据： 8.3.2(18)（第 131 页）。

8.3.2-05 未参与触发的可分派端口不自动冻结

其他具有 dispatch 触发能力而未参与本次触发的端口，默认不随本次 dispatch 冻结；可通过显式 Receive_Input 请求取得其输入。

依据： 8.3.2(20)（第 131 页）；8.3.2(21)（第 131–132 页）。

8.3.2-06 其余端口采用规定输入时刻

在本次分派触发端口及其他可触发 dispatch 的端口之外，其余输入端口仍按各自有效 Input_Time 冻结。

依据： 8.3.2(20)（第 131 页）；8.3.2(21)（第 131–132 页）。

8.3.2-07 本次分派触发端口随分派冻结

指定 dispatch 条件时，满足该条件的 event/event data 端口组合确定本次触发端口；这些端口的输入作为本次 dispatch 的一部分被冻结。

依据： 8.3.2(21)（第 131 页）；8.3.2(21) 续（第 132 页）。

8.3.2-08 匹配当前模式的端口事件请求转换

event 或 event data 到达被组件模式转换命名的端口时，依模式切换语义处理；只有命中当前模式出转换的触发源才请求该转换，未命中时不改变该组件模式。

依据： 8.3.2(22)（第 132 页）；9.2.4(33)（第 164 页）；12(16)（第 216 页）。

8.3.2-09 普通 data 默认完成时发送

未指定其他有效输出时刻、且不适用 immediate 或 delayed 专门时序的普通 data 端口，在执行完成时发送输出。

依据： 8.3.2(23)（第 132 页）；8.3.2(25) Completion（第 132 页）。

8.3.2-10 Start 偏移内发送输出

端口有效 Output_Time 为 Start 时，在自实际执行开始累计的执行时间落入所指定正偏移区间时发送输出；immediate 或 delayed 连接另受其专门时序约束。

依据： 8.3.2(25) Start（第 132 页）。

8.3.2-11 Completion 偏移内发送输出

端口有效 Output_Time 为 Completion 时，按相对执行完成的执行时间偏移区间发送输出：负偏移在完成前，零偏移在完成时；适用连接专门时序时遵循其约束。

依据： 8.3.2(25) Completion（第 132 页）。

8.3.2-12 Deadline 输出采用时钟期限

端口有效 Output_Time 为 Deadline 时，在 dispatch 对应的时钟 deadline 发送输出，发送时刻不由已累计的处理器执行时间决定。

依据： 8.3.2(25) Deadline（第 132 页）。

8.3.2-13 NoIO 禁止新输出发送

端口有效 Output_Time 为 NoIO 时，不从该端口发送源程序的新输出；该属性可按模式使端口进入或离开此禁止发送状态。

依据： 8.3.2(25) NoIO（第 132 页）。

8.3.2-14 Dynamic 允许执行中任意输出时刻

端口有效 Output_Time 为 Dynamic 时，输出可在执行期间任意时刻发生，不指定相对 Start 或 Completion 的固定偏移。

依据： 8.3.2(25) Dynamic（第 132 页）。

8.3.2-15 输出列表给出多个发送机会

Output_Time 是多项列表时，各项分别表示可以发送输出的时刻或时间区间；该列表不强制每次 dispatch 只有一个输出时点。

依据： 8.3.2(26)（第 132 页）。

8.3.2-16　运行时可以执行完成和期限发送

完成时或 deadline 的规定输出可由运行时自动传输，也可由源程序的 Send_Output 调用传输；发送仍须符合端口及连接的有效时间约束。

依据： 8.3.2(27)（第 132 页）。

8.3.2-17　输入频率按单位解释

Input_Rate 给出端口每次 dispatch 或每秒的预期输入次数范围及分布，默认每次 dispatch 固定一次，未指定分布时为 Fixed；Input_Time 为 NoIO 时不应用该输入频率。

依据： 8.3.2(28)（第 132 页）；8.3.2(29)（第 132 页）；A.4 Input_Rate、Rate_Spec（第 260–261 页）。

8.3.2-18　输出频率按单位解释

Output_Rate 给出端口每次 dispatch 或每秒的预期输出次数范围及分布，默认每次 dispatch 固定一次，未指定分布时为 Fixed；Output_Time 为 NoIO 时不应用该输出频率。

依据： 8.3.2(28)（第 132 页）；8.3.2(29)（第 132 页）；A.4 Output_Rate、Rate_Spec（第 261 页）。

8.3.3　队列与扇出策略

8.3.3-01　事件接收队列的缺省容量

线程、设备和处理器的 incoming event 及 event data 端口默认具有容量为 1 的接收队列；显式 Queue_Size 替换该默认容量。

依据： 8.3.3(30)（第 133 页）；9.2.6(53)（第 166 页）；A.4 Queue_Size（第 260 页）。

8.3.3-02　活动接收者把普通事件加入队列

语义连接最终目标组件正在活动处理、且接收端口没有被模式转换专门处理等例外时，最终源交付的 event 或 event data 加入该端口接收队列。

依据： 8.3.3(30)（第 133 页）；8.3.4(38)（第 134 页）。

8.3.3-03　满队列丢弃最旧项

队列已达到 Queue_Size 而新 event 或 event data 到达时，Overflow_Handling_Protocol 为 DropOldest 则删除最旧项并接纳新项；未指定该属性时采用此策略。

依据： 8.3.3(31) DropOldest（第 133 页）；A.4 Overflow_Handling_Protocol（第 260 页）。

8.3.3-04　满队列丢弃新到项

队列已达到 Queue_Size 而新 event 或 event data 到达时，Overflow_Handling_Protocol 为 DropNewest 则舍弃新到项，队列已有项不因这次到达而被替换。

依据： 8.3.3(31) DropNewest（第 133 页）；A.4 Overflow_Handling_Protocol（第 260 页）。

8.3.3-05 满队列 Error 引起线程恢复

队列已达到 Queue_Size 而新 event 或 event data 到达时，Overflow_Handling_Protocol 为 Error 则对接收线程产生错误并调用其错误恢复。

依据： 8.3.3(31) Error（第 133 页）；A.4 Overflow_Handling_Protocol（第 260 页）。

8.3.3-06 溢出错误可定位端口

端口队列按 Overflow_Handling_Protocol=Error 产生错误后，线程可通过 Get_Error_Code 确定引起该错误的端口。

依据： 8.3.3(31)（第 133 页）。

8.3.3-07 单队列默认先入先出

单个事件队列按 Queue_Processing_Protocol 服务，默认 FIFO，先到达的项先被服务；实现支持且显式采用其他协议时，按所选协议排序。

依据： 8.3.3(32)（第 133 页）；A.4 Queue_Processing_Protocol（第 260 页）；A.8 Supported_Queue_Processing_Protocols（第 276 页）。

8.3.3-08 多触发队列优先服务高 Urgency

aperiodic、sporadic、timed 或 hybrid 线程的多个 dispatch 触发队列同时非空时，先服务 Urgency 较高的端口；具有多个 dispatch 触发端口的目标设备同样按该规则选择。

依据： 8.3.3(32)（第 133 页）；9.2.6(57)（第 167 页）。

8.3.3-09 相同 Urgency 跨端口统一选项

多个非空 dispatch 触发端口具有相同 Urgency 时，按其共同 Queue_Processing_Protocol 选择待服务项；FIFO 选择这些端口中最早到达的事件，已选择其他受支持算法时按该算法处理。

依据： 8.3.3(32)（第 133 页）。

8.3.3-10 OneItem 仅出队一个冻结项

Dequeue_Protocol 为 OneItem 时，非空冻结队列只出队一个项并提供给源程序，Next_Value 不改变所提供的项；空队列不产生新项。未指定 Dequeue_Protocol 时采用 OneItem。

依据： 8.3.3(33) OneItem（第 133 页）；A.2 Dequeue_Protocol（第 251 页）。

8.3.3-11 AllItems 一次取走全部可用冻结项

Dequeue_Protocol 为 AllItems 时，一次出队全部可用冻结项，源程序通过 Next_Value 逐项访问；未读取的冻结项不保留到下一输入时刻，空队列不产生新项。已设置 Dequeued_Items 时，可供应用访问的项数还受该上限约束。

依据： 8.3.3(33) AllItems（第 133 页）；A.2 Dequeue_Protocol（第 251 页）。

8.3.3-12 MultipleItems 逐项出队并保留剩余项

Dequeue_Protocol 为 MultipleItems 时，Next_Value 按次从冻结队列出队并提供下一项；未出队的项留到下次 dispatch 或 Receive_Input，空队列不产生新项。已设置 Dequeued_Items 时，本次可提供项数还受该上限约束。

依据： 8.3.3(33) MultipleItems（第 133 页）；A.2 Dequeue_Protocol（第 251 页）。

8.3.3-13 Broadcast 交付全部接收者

具有多个出连接的端口选择 Fan_Out_Policy=Broadcast 时，本次输出发送给全部接收者。

依据： 8.3.3(35) Broadcast（第 133 页）；A.4 Fan_Out_Policy（第 259 页）。

8.3.3-14 RoundRobin 轮流选择接收者

具有多个出连接的端口选择 Fan_Out_Policy=RoundRobin 时，每次按顺序选择一个接收者，使输出均匀轮流分配。

依据： 8.3.3(35) RoundRobin（第 133 页）；A.4 Fan_Out_Policy（第 259 页）。

8.3.3-15 Selective 按内容选择接收者

具有多个出连接的端口选择 Fan_Out_Policy=Selective 时，按内容或路由信息把本次输出交给一个接收者。

依据： 8.3.3(35) Selective（第 133 页）；A.4 Fan_Out_Policy（第 259 页）。

8.3.3-16 OnDemand 选择等待分派的接收者

具有多个出连接的端口选择 Fan_Out_Policy=OnDemand 时，将待服务输出交给下一个在端口等待输入且可被 dispatch 的接收者。

依据： 8.3.3(35) OnDemand（第 133 页）；A.4 Fan_Out_Policy（第 259 页）。

8.3.3-17 即时扇出策略不为分发建立队列

Fan_Out_Policy 为 Broadcast、RoundRobin 或 Selective 时，扇出过程直接传递数据或事件，不因该分发策略另行排队。

依据： 8.3.3(35)（第 133 页）。

8.3.3-18 OnDemand 使用接收者服务的共享队列

Fan_Out_Policy 为 OnDemand 时，扇出端口使用由接收者服务的共享队列；容量及其他队列特征由该扇出端口的属性确定。

依据： 8.3.3(35)–(36)（第 133 页）；8.3.3(36) 续（第 134 页）。

8.3.4 事件与子程序

8.3.4-01 一次分派可显式发送多个事件

线程一次 dispatch 执行期间可通过 Send_Output 发送零个或多个 event 及 event data，不因一次 dispatch 而被限制为恰好发送一个事件。

依据： 8.3.4(37)（第 134 页）。

8.3.4-02 完成输出只提交尚未发送的新事件数据

在完成时进行 event data 输出且符合端口有效 Output_Time 的情况下，提交端口中已更新、尚未通过显式 Send_Output 发送的新值；已发送且未再次更新的值不属于本次待提交集合。

依据： 8.3.4(37)（第 134 页）；8.3.2(24)–(27)（第 132 页）。

8.3.5 端口运行时服务

8.3.5-01 Send_Output 按指定列表启动传输

Send_Output 在参数指定的发送时间启动所列 outgoing 端口的 data、event 或 event data 传输；未列端口不因该调用一并发送。

依据： 8.3.5(41)（第 135 页）；A.9(3)（第 279 页）。

8.3.5-02 同次 Send_Output 逻辑同时发送

一次 Send_Output 调用指定多个输出端口时，这些端口的发送在逻辑上同时发生。

依据： 8.3.5(41)（第 135 页）；A.9(3)（第 279 页）。

8.3.5-03 Put_Value 产生待发送值

Put_Value 提供给端口变量的值，在后续 Send_Output 或运行时规定的完成/deadline 输出时刻传输；单独调用 Put_Value 不表示该值已完成发送。

依据： 8.3.5(42)（第 135 页）；A.9(4)（第 279 页）。

8.3.5-04 data 的 Put_Value 覆盖前值

对 data 端口调用 Put_Value 时，新设置值覆盖该端口此前设置的值。

依据： 8.3.5(43)a（第 135 页）；A.9(5)a（第 279 页）。

8.3.5-05 事件类 Put_Value 按队列属性入队

对 event 或 event data 端口连续调用 Put_Value 时，各次提交按该端口所配置的队列属性入队。

依据： 8.3.5(43)b（第 135 页）；A.9(5)b（第 279 页）。

8.3.5-06 Receive_Input 覆盖指定端口的可见输入

Receive_Input 在参数指定的参考输入时间冻结所列 incoming 端口，并以本次内容覆盖其端口变量先前内容，使本次输入可供源程序访问；未列端口不因该调用重新冻结。

依据： 8.3.5(44)（第 135 页）；A.9(6)（第 279–280 页）。

8.3.5-07 data 的 Get_Count 返回新鲜度计数

对 data 端口调用 Get_Count，值已更新则返回 1，否则返回 0；返回 0 不表示此前数据值必已消失。

依据： 8.3.5(45)a（第 136 页）；A.9(7)a（第 280 页）。

8.3.5-08 Get_Value 返回当前端口值

Get_Value 返回所指定端口变量当前的数据值。

依据： 8.3.5(46)（第 136 页）；A.9(8)（第 280 页）。

8.3.5-09 Get_Value 读取不推进队列

Get_Value 本身不出队或前移当前项；没有 Receive_Input、Next_Value 或其他更新介入时，重复调用得到同一值。

依据： 8.3.5(46)（第 136 页）；A.9(8)（第 280 页）。

8.3.5-10 data 取值无需 Next_Value

data 端口的当前值可直接获取，不需要先调用 Next_Value。

依据： 8.3.5(47)a（第 136 页）；A.9(9)a（第 280 页）。

8.3.5-11 首个事件数据值立即可读

event data 端口的首个可用值可直接通过 Get_Value 访问，无须先调用 Next_Value。

依据： 8.3.5(47)b（第 136 页）；A.9(9)b（第 280 页）。

8.3.5-12 Next_Value 使后续项成为当前值

event data 端口存在后续可用项且出队协议允许前移时，Next_Value 把下一项变成端口变量当前值；OneItem 协议不允许该调用推进。

依据： 8.3.5(47)b（第 136 页）；8.3.3(33)（第 133 页）；A.9(9)b（第 280 页）。

8.3.5-13 Updated 判断两次接收之间的新到达

Updated 查询自上次 Receive_Input 以来是否有新输入传送到该端口，有则为真、否则为假；端口保存着旧值本身不使该结果为真。

依据： 8.3.5(48)（第 136 页）；A.9(10)（第 281 页）。

8.3.5-14 隐式中间队列遵循端口队列语义

实现为计算模型或架构风格需要而在最终源与目标之间隐式增加中间队列时，这些队列同样遵循本节的容量、溢出及服务语义。隐式增加队列属于实现许可。

依据： 8.3.5(49)（第 137 页）。

8.4 子程序与子程序组访问

8.4-01 远程代理交换请求与结果

远程调用者先调用本地子程序代理；代理按需编组、解组参数，并与远程服务交换执行请求和返回结果，实际被调代码仍由远程服务执行。

依据： 8.4(6)（第 140 页）。

8.4-02 客户端代理时间单独计量

Client_Subprogram_Execution_Time 确定远程调用客户端代理部分的执行时间范围，不包含服务线程执行远程子程序的时间。

依据： 8.4(6)（第 140 页）；A.3 Client_Subprogram_Execution_Time（第 254 页）。

8.4-03 远程子程序事件映射到调用代理

远程被调子程序发出的 event 或 event data 映射到调用者本地子程序代理的对应 event 或 event data 端口。

依据： 8.4(7)（第 140 页）。

8.4-04 忙服务线程的远程请求排队

服务线程忙于当前分派时，新远程请求等待；实现用入口队列表示待服务请求时，采用事件端口队列语义。待执行子程序对已锁定共享数据的访问另依资源等待规则。

依据： 8.4(8)–(9)（第 140 页）；8.4(8)（第 140 页）；A.4 Queue_Size（第 260 页）。

8.4-05 一次服务分派只执行一个远程入口

同一服务线程即使声明多个 provides subprogram access 入口，每次 dispatch 也只执行其中一个被请求子程序。

依据： 8.4(8)（第 140 页）。

8.4-06 子程序调用频率的计量

Subprogram_Call_Rate 按每次所属线程 dispatch 或每秒解释预期调用次数范围及分布；默认每次 dispatch 固定一次，未指定分布时为 Fixed。

依据： 8.4(10)（第 140 页）；8.4 Standard Properties（第 139 页）；A.4 Subprogram_Call_Rate（第 261–262 页）。

8.5 子程序参数

8.5-01 in out 参数按值传入

调用开始时，in out parameter 以值作为子程序输入；它不因 in out 声明而取得对调用方存储的直接共享引用，引用访问用 requires data access 表示。

依据： 8.5(3)（第 144 页）。

8.5-02 in out 参数按值传回

调用返回时，in out parameter 以值向调用方提供输出；需要共享同一数据对象时须使用 requires data access，不将该按值返回解释为引用共享。

依据： 8.5(3)（第 144 页）。

8.6 数据访问

8.6-01 读访问的数据来自共享对象

数据访问具有读权限或其有向连接从 data 组件指向 requires data access 时，读取的数据从共享 data 流向要求访问的组件。

依据： 8.6(3)（第 146 页）；9.4(9)（第 173 页）。

8.6-02 写访问的数据流向共享对象

数据访问具有写权限或其有向连接指向 data 组件时，写入数据从要求访问的组件流向该共享 data。

依据： 8.6(3)（第 146 页）；9.4(9)（第 173 页）。

8.6-03 抽象 in 特征细化后只读

data access 由方向为 in 的抽象 feature 细化而来时，该继承方向把 data 访问限制为只读。

依据： 8.6(5)（第 146 页）。

8.6-04 抽象 out 特征细化后只写

data access 由方向为 out 的抽象 feature 细化而来时，该继承方向把 data 访问限制为只写。

依据： 8.6(5)（第 146 页）。

8.6-05 Access_Time 确定访问窗口

Access_Time 的 First 和 Last 确定组件需要共享 data 访问的执行时间窗口；缺省从执行开始零偏移至执行完成零偏移，覆盖整个执行期。

依据： 8.6(7)（第 146 页）；A.5 Access_Time（第 263 页）。

8.6-06 共享 data 读取实际对象的当前值

通过代表共享 data 的变量或 Get_Value 读取时，取得该共享 data 实例当前的值。

依据： 8.6(7)（第 146 页）。

8.6-07 共享 data 写入立即生效

通过代表共享 data 的变量或 Put_Value 写入时，立即更新该共享 data 实例，不等待端口 Send_Output 或线程完成才提交。

依据： 8.6(7)（第 146 页）。

8.7 总线访问

8.7-01 bus 访问解析到同一共享总线

requires bus access 沿访问连接解析到实际 bus 子组件；provides bus access 暴露内部 bus。多个访问解析到同一 bus 时，共享该通信资源而不复制为各自独立的 bus。

依据： 8.7(3)–(5)（第 149 页）。

8.8 虚拟总线访问

8.8-01 virtual bus 访问解析到实际实例

requires 与 provides virtual bus access 沿 virtual bus access 连接解析到实际 virtual bus 子组件；provides 特征向外开放该内部实例，各使用者不因各自访问特征而得到无关副本。

依据： 8.8(4)（第 151 页）；8.8(2)–(3)（第 151 页）。

8.9 内部特征与处理器代理特征

8.9-01 self 端点引用组件内部事件源

连接中的 self 限定名解析到当前组件实现声明的 internal event 或 event data，表示组件自身内部事件源，不生成同名子组件。

依据： 8.9(8)（第 151 页）。

8.9-02 processor 代理跟随实际处理器绑定

组件实现的 processor feature 代理其所绑定 processor 的端口或子程序特征；processor 限定的连接端点解析到该处理器的对应特征，不新建一个处理器。

依据： 8.9(9)（第 151 页）；9.2.1(15)（第 160 页）。

第 9 章　连接

9-01　连接链形成最终端点间的实例

实例化已细化到线程的模型时，沿包含层次追踪连接声明链，为最终源到最终目标建立语义连接；中间层次接口映射不成为独立通信终点。

依据： 9(2)（第 152 页）。

9-02　未细化连接以当前叶组件为端点

模型尚未细化到线程时，连接实例的最终源和目标取当前系统层次的叶组件，不补造尚未声明的内部线程端点。

依据： 9(2)（第 152 页）。

9.1　特征连接

9.1-01　有向抽象特征确定源目标

两个抽象特征连接且给定特征方向时，out 特征作为数据流源，in 特征作为目标；实际流向同时满足连接声明的方向。

依据： 9.1(1)（第 153 页）。

9.1-02　未限方向的抽象连接双向解释

抽象特征及其连接均没有单向限制时，该特征连接按双向解释，不能仅按端点文字排列固定为单向。

依据： 9.1(1)（第 153 页）。

9.1-03　嵌套成员连接独立于分组深度

特征连接可直接解析到子组件 feature group 内的嵌套成员；两端分组深度不同不改变所引用的实际成员端点。

依据： 9.1(2)（第 153 页）。

9.1-04　有向声明限制语义连接流向

语义 feature 连接中存在有向连接声明时，实际传输方向限制在该声明与端点共同允许的方向；双向段不取消该单向限制。

依据： 9.1(L5)–(L6)（第 154 页）。

9.1-05　组连接和成员连接分别生成连接

同一成员同时具有独立特征连接和其外层 feature group 连接时，两者分别产生语义连接，不能因共享成员而覆盖其中一条。

依据： 9.1(4)（第 155 页）。

9.2 端口连接

9.2-01 共享 data 可被端口采样

实现支持共享 data 作为语义端口连接源时，该 data 的内容可被连接的 data 或 event data 端口采样为输入；连接只有这一端是 data 组件。

依据： 9.2(5)（第 156 页）；9.2.6(63)（第 167 页）。

9.2-02 端口输出可更新 data 组件

语义端口连接以 data 组件为最终目标、以 data 或 event data 端口为源时，端口输出更新目标 data 组件；不把该连接解释为两个 data 组件之间的连接。

依据： 9.2(5)（第 156 页）。

9.2-03 可双向端点允许两个源方向

双向 port 连接的两端均具相应输入输出能力时，每端都可作为源，形成相反的两个允许传输方向。

依据： 9.2(L2)（第 158 页）；9.2.2(23)（第 162 页）。

9.2-04 双向连接仍受端点方向限制

双向 port 连接的端点为单向 port 或受 Access_Right 限制的 data access 时，实际数据流仅采用端点允许的方向，连接符号不扩大端点能力。

依据： 9.2(L2)、(L9)（第 158 页）。

9.2-05 Equivalence 保持相同数据表示

合法数据连接使用 Equivalence 匹配时，匹配分类器按所声明等价关系视为同一数据表示，不执行数据类型转换。

依据： 9.2(L14) Equivalence（第 159 页）。

9.2-06 Subset 只映射选定数据元素

合法数据连接使用 Subset 匹配且所绑定 bus 或 virtual bus 支持该匹配时，将源数据中的指定子集映射为目标的全部元素；具体映射由已声明的匹配关系确定。

依据： 9.2(L14) Subset（第 159 页）。

9.2-07 Conversion 执行受支持的数据转换

合法数据连接使用 Conversion 匹配且所绑定 bus 或 virtual bus 支持该转换时，将源分类器的数据转换为目标分类器的数据；不由分类器名称猜测未声明的转换。

依据： 9.2(L14) Conversion（第 159 页）。

9.2.1 端口连接特征

9.2.1-01 feature 连接细化为端口连接

通用 feature connection 合法细化为 port connection 后，原连接按端口连接类别解释，不另留一个旧类别的平行连接。

依据： 9.2.1(10)（第 160 页）。

9.2.1-02 双向连接细化后收缩为单向

继承的双向 port connection 合法细化为单向连接后，该连接仅按细化后的指定方向通信，不继续保留旧的反向传输方向。

依据： 9.2.1(10)（第 160 页）。

9.2.1-03 普通模式仅使用其活动端口连接

稳定模式期间仅在当前模式活动的 port 连接传输；仅属其他模式或仅属模式转换的连接不因源输出而传输。未带 in modes 的连接属于全部普通模式。

依据： 9.2.1(12)（第 160 页）；12(13)（第 216 页）。

9.2.1-04 模式转换连接传递数据

数据端口连接被声明为在当前模式转移中活动，且源线程在进入 mode_transition_in_progress 时已完成执行时，在转移期传递源 out 数据端口值，使其在接收线程首次分派时可用。

依据： 12(13)（第 216 页）；13.6(13)（第 229 页）；9.2.1(13)（第 160 页）；13.6(13)–(14)（第 229 页）。

9.2.1-05 转换连接转移事件队列内容

event 或 event data 连接显式属于所发生模式转换时，在实际切换时转移其队列内容，以便在不同模式活动的组件间传递该状态。

依据： 9.2.1(13)（第 160 页）。

9.2.1-06 平台 data 源在目标分派时采样

processor 或 device 作为 data 连接源时，在目标线程或设备 dispatch 时启动并完成传输，可据此采样相应平台寄存器值。

依据： 9.2.1(15)（第 160 页）。

9.2.1-07 平台中断启动事件传输

processor 或 device 作为 event 连接源时，中断发生启动相应事件传输。

依据： 9.2.1(15)（第 160 页）。

9.2.1-08　event 目标把输出识别为事件

合法连接把源 event、data、event data 输出或 data 写入送到 event 端口时，目标将其作为事件排队，不携带源数据载荷。

依据： 9.2.1(16) event port（第 160 页）；8.3(3)c（第 128 页）。

9.2.1-09　event data 目标排队接收消息

合法连接把携带数据的输出或 data 写入送到 event data 端口时，目标把数据接收为可排队的 event data。

依据： 9.2.1(16) event data port（第 161 页）。

9.2.1-10　data 目标只形成最近接收值

合法连接把 data、event data 输出或 data 写入送到 data 端口时，目标变量取得最近收到的数据值，不因源具有消息队列而建立接收消息队列。

依据： 9.2.1(16) data port（第 161 页）。

9.2.2　端口连接拓扑

9.2.2-01　事件扇入形成共同接收序列

多个源通过入连接向同一 event 或 event data 端口交付时，来自各源的项进入该端口的接收序列，并按其队列规则处理。

依据： 9.2.2(18)（第 161 页）。

9.2.2-02　公共端口扇入扇出展开全部端点对

多个源经同一公共 event 或 event data 端口扇入，再由公共路径扇出到多个目标时，结构上为每个最终源与每个最终目标形成语义连接；实际交付对象仍服从 Fan_Out_Policy。

依据： 9.2.2(19)（第 162 页）。

9.2.2-03　经特征组分离的路径按成员配对

多个源和目标通过特征组中分别的成员路径相连时，按对应成员形成最终源与目标的成对语义连接，不因存在一个组连接就扩展为所有成员间的全连接。

依据： 9.2.2(19)（第 162 页）。

9.2.2-04　聚合输出等待最晚源完成

周期线程向可外部访问的聚合 data 端口元素提供输出时，每次聚合取 dispatch 对齐的源线程；在这些源全部完成后发送聚合值，发送发起时刻为其最大完成时刻 max(t_complete)。

依据： 9.2.2(21)（第 162 页）。

9.2.2-05　接收者可取得聚合数据的单个元素

聚合 data 端口的 data 分类器通过 provides data access 公开元素时，指向某元素的连接使接收者取得该元素，不自动取得未连接的其他元素。

依据： 9.2.2(21)（第 162 页）。

9.2.2-06　接收者可取得聚合数据子集

聚合 data 端口的接收者使用受支持的 Subset 等匹配关系时，收到该匹配规定的元素子集，不把整个源聚合值强加为目标数据。

依据： 9.2.2(21)（第 162 页）。

9.2.3　组件数组与特征数组的连接模式

9.2.3-01　组件数组维度由内向外合成

语义连接端点的有效组件数组维度由端点自身及连接涉及的各层包含组件数组维度相加得到，排列顺序自端点组件向包含层次外侧。

依据： 9.2.3(24)（第 162 页）。

9.2.3-02　特征数组维度排在最前

语义连接端点为 feature array 时，将其维度放在有效数组维度序列最前；Connection_Pattern 和 Connection_Set 据此序列定位实例。

依据： 9.2.3(25)（第 162 页）。

9.2.3-03　All_To_All 连接全部元素对

Connection_Pattern 的某一维为 All_To_All 时，该维每个源元素连接每个目标元素；两端该维大小可以不同。

依据： 9.2.3(26) All_To_All（第 162 页）；A.8 Supported_Connection_Patterns（第 275 页）。

9.2.3-04　One_To_One 连接同索引元素

源与目标同维且各对应维大小相同时，One_To_One 在该维仅连接相同索引的元素。

依据： 9.2.3(26) One_To_One（第 162 页）；A.8 Supported_Connection_Patterns（第 275 页）。

9.2.3-05　Next 连接后一项而不回绕

两端对应数组维大小相同时，Next 使源索引 i 连接目标 i+1；源末项没有该模式生成的连接。

依据： 9.2.3(26) Next（第 163 页）；A.8 Supported_Connection_Patterns（第 275 页）。

9.2.3-06　Previous 连接前一项而不回绕

两端对应数组维大小相同时，Previous 使源索引 i 连接目标 i−1；源首项没有该模式生成的连接。

依据： 9.2.3(26) Previous（第 163 页）；A.8 Supported_Connection_Patterns（第 275 页）。

9.2.3-07　各维连接模式共同确定端点对

给定一个逐维模式组合时，一条元素连接须同时满足该组合各维的连接关系；例如两维均为 Next 时，连接的是沿两维同时后移的对角邻居。

依据： 9.2.3(26)–(27)（第 163 页）；9.2.3 Figure 15（第 163 页）。

9.2.3-08　Cyclic_Next 在末端回到首项

两端对应数组维大小相同时，Cyclic_Next 把各源项连到后一目标项，并把源末项连接到目标首项。

依据： 9.2.3(26) Cyclic_Next（第 163 页）；A.8 Supported_Connection_Patterns（第 275 页）。

9.2.3-09　Cyclic_Previous 在首端回到末项

两端对应数组维大小相同时，Cyclic_Previous 把各源项连到前一目标项，并把源首项连接到目标末项。

依据： 9.2.3(26) Cyclic_Previous（第 163 页）；A.8 Supported_Connection_Patterns（第 275 页）。

9.2.3-10　One_To_All 复制到额外目标维

目标比源多一个对应维度且该维模式为 One_To_All 时，把其他维度已确定的每条连接复制到这一目标维的全部元素。

依据： 9.2.3(26) One_To_All（第 163 页）；A.8 Supported_Connection_Patterns（第 275 页）。

9.2.3-11　All_To_One 汇入一个目标

源比目标多一个对应维度且该维模式为 All_To_One 时，把这一源维的各元素连接到其他维度所确定的同一目标。

依据： 9.2.3(26) All_To_One（第 163 页）；A.8 Supported_Connection_Patterns（第 275 页）。

9.2.3-12　Even_To_Even 只配对偶数索引

项目的 Supported_Connection_Patterns 已纳入并合法选用该模式时，Even_To_Even 只连接源与目标中索引相同的偶数项，其他索引不因该模式产生连接。

依据： 9.2.3(26) Even_To_Even（第 163 页）；Appendix A(2)（第 241 页）；A.8(1)–(2)（第 274 页）。

9.2.3-13　Odd_To_Odd 只配对奇数索引

项目的 Supported_Connection_Patterns 已纳入并合法选用该模式时，Odd_To_Odd 只连接源与目标中索引相同的奇数项，其他索引不因该模式产生连接。

依据： 9.2.3(26) Odd_To_Odd（第 163 页）；Appendix A(2)（第 241 页）；A.8(1)–(2)（第 274 页）。

9.2.3-14　Next_Next 向后偏移两项

项目的 Supported_Connection_Patterns 已纳入并合法选用该模式时，两端对应维大小相同时，Next_Next 连接源 i 与目标 i+2；源末两项不产生越界连接。

依据： 9.2.3(26) Next_Next（第 163 页）；Appendix A(2)（第 241 页）；A.8(1)–(2)（第 274 页）。

9.2.3-15 Previous_Previous 向前偏移两项

项目的 Supported_Connection_Patterns 已纳入并合法选用该模式时，两端对应维大小相同时，Previous_Previous 连接源 i 与目标 i−2；源首两项不产生越界连接。

依据： 9.2.3(26) Previous_Previous（第 163 页）；Appendix A(2)（第 241 页）；A.8(1)–(2)（第 274 页）。

9.2.3-16 Cyclic_Next_Next 向后偏移并回绕

项目的 Supported_Connection_Patterns 已纳入并合法选用该模式时，两端对应维大小相同时，Cyclic_Next_Next 向后偏移两项连接，并将源末两项依次环回到目标首两项。

依据： 9.2.3(26) Cyclic_Next_Next（第 163 页）；Appendix A(2)（第 241 页）；A.8(1)–(2)（第 274 页）。

9.2.3-17 Cyclic_Previous_Previous 向前偏移并回绕

项目的 Supported_Connection_Patterns 已纳入并合法选用该模式时，两端对应维大小相同时，Cyclic_Previous_Previous 向前偏移两项连接，并将越过首端的两项环回到目标对应末项。

依据： 9.2.3(26) Cyclic_Previous_Previous（第 163 页）；Appendix A(2)（第 241 页）；A.8(1)–(2)（第 274 页）。

9.2.3-18 多个模式组合取连接并集

Connection_Pattern 给出多个逐维模式组合时，最终保留各组合生成的全部连接关系，不以后一组合替换前一组合。

依据： 9.2.3(27)（第 163 页）；9.2.3 Figure 15（第 163 页）。

9.2.3-19 Connection_Set 直接指定元素对

Connection_Set 的每个 (src,dst) 对确定一条源数组元素到目标数组元素的语义连接；各索引向量按有效维度逐维给出索引，索引从 1 开始。

依据： 9.2.3(28)（第 164 页）；A.4 Connection_Set、Connection_Pair（第 259 页）。

9.2.3-20 显式连接集与模式取并集

同时给出 Connection_Set 与 Connection_Pattern 时，最终语义连接集为显式索引配对和模式展开连接的并集。

依据： 9.2.3(29)（第 164 页）。

9.2.4 连接传输的时机

9.2.4-01 push 由最终源发起数据传输

最终目标为 data 端口且 Transmission_Type 为 push 时，由最终源启动实际传输；未指定 Transmission_Type 时采用这一发起方式。

依据： 9.2.4(34)（第 164 页）；A.4 Transmission_Type（第 260 页）。

9.2.4-02　pull 由最终目标请求数据传输

最终目标为 data 端口且 Transmission_Type 为 pull 时，由最终目标请求并启动实际传输，不仅因源产生数据就启动 push 传输。

依据： 9.2.4(34)（第 164 页）；A.4 Transmission_Type（第 260 页）。

9.2.5　采样、即时与延迟数据端口通信

9.2.5-01　sampled 在接收输入时刻采样

周期目标线程或设备的 data 输入采用 sampled 连接时，在 dispatch 或有效 Input_Time 指定时刻取当时可用输出，不等待发送者本次 dispatch 或计算完成；因并发和抢占可得到不同样本。Timing 缺省为 sampled。

依据： 9.2.5(38)（第 165 页）；9.2 Standard Properties: Timing（第 159 页）；A.4 Timing（第 260 页）。

9.2.5-02　immediate 接收执行等待源输出

最终源、目标均为周期线程或周期设备且最终目标为 data 端口时，immediate 连接中，与源同时 dispatch 或源 dispatch 后首个目标 dispatch 对应的接收执行等待本次源输出；默认等待源完成，指定受支持的唯一更早 Output_Time 时改为等待该输出时刻。

依据： 9.2.5(40)（第 165 页）；9.2.5(44)、(47)（第 165–166 页）。

9.2.5-03　同步域内保证确定性采样

最终源、目标均为周期线程或周期设备且最终目标为 data 端口时，同一同步域中的 immediate 或 delayed 连接须保证其规定的确定性采样。

依据： 9.2.5(42)（第 165 页）；9.2.6(64)（第 168 页）。

9.2.5-04　跨域确定性依赖协调协议

最终源、目标均为周期线程或周期设备且最终目标为 data 端口时，跨异步同步域的 immediate 或 delayed 连接，仅在运行时提供处理时钟漂移的逻辑协调协议时保证确定性采样；实现允许不支持这类跨域连接。

依据： 9.2.5(42)（第 165 页）；9.2.6(64)（第 168 页）。

9.2.5-05　delayed 在首次不早于源 deadline 的分派接收

最终源、目标均为周期线程或周期设备且最终目标为 data 端口时，delayed 数据在目标第一次不早于源 deadline 的 dispatch 可见，目标输入按 Dispatch 解释；源 deadline 与目标 dispatch 逻辑同时发生时，传输在同一逻辑时刻内完成。

依据： 9.2.5(45)（第 165 页）；9.2.5(48)、(50)（第 166 页）。

9.2.5-06　immediate 在源输出时传送数据

最终源、目标均为周期线程或周期设备且最终目标为 data 端口时，immediate 连接默认在源完成时把源值传送到接收端；显式指定唯一的更早 Output_Time 时，在该输出时刻传送，作为相应接收执行的输入。

依据： 9.2.5(47)（第 166 页）；9.2.5(44)、(49)（第 165–166 页）。

9.2.5-07　immediate 链保持传递的先后关系

最终源、目标均为周期线程或周期设备且最终目标为 data 端口时，多个 immediate 语义连接串联时，等待和传输所建立的源先于目标的关系沿连接链传递，不只约束相邻的一段。

依据： 9.2.5(47)（第 166 页）。

9.2.5-08　immediate 通信受目标 deadline 约束

最终源、目标均为周期线程或周期设备且最终目标为 data 端口时，immediate 连接默认要求源与目标均不晚于目标 deadline 完成；采用唯一的更早 Output_Time 时，该约束中的源完成时刻改用源输出时刻，目标完成仍不得超过目标 deadline。

依据： 9.2.5(47)（第 166 页）。

9.2.5-09　delayed 在源 deadline 启动传输

最终源、目标均为周期线程或周期设备且最终目标为 data 端口时，delayed 连接在源的时钟 deadline 启动数据传输，源输出按 Deadline 解释；源提前完成不使该数据提前按 delayed 语义可用。

依据： 9.2.5(48)–(50)（第 166 页）；9.2.5(41),(45)（第 165 页）。

9.2.5-10　immediate 强制 Start 零偏移输入

最终源、目标均为周期线程或周期设备且最终目标为 data 端口时，immediate 连接的目标输入按 Start 零偏移解释，忽略另行指定的其他 Input_Time。

依据： 9.2.5(49)（第 166 页）。

9.2.6　语义端口连接与端口队列

9.2.6-01　输出先进入首个多出边 OnDemand 队列

沿语义端口连接传送输出时，数据或事件加入路径上遇到的第一个采用 OnDemand 且具有多条出连接的端口队列。

依据： 9.2.6(55)（第 167 页）。

9.2.6-02　单出边 OnDemand 端口允许直接通过

路径上的 OnDemand 端口只有一条出连接时，实现可直接传递该输出而不在该端口排队。

依据： 9.2.6(55)（第 167 页）。

9.2.6-03　未直接接收的目标服务末端有效队列

最终目标不能直接由语义连接取得输入时，服务声明链上最后一个满足 OnDemand、多出连接且有语义连接向其队列供数的端口队列。

依据： 9.2.6(56)（第 167 页）。

9.2.6-04　单一上游 OnDemand 来源由上游供数

一个 OnDemand 端口仅从另一个 OnDemand 端口取得输入时，服务那个上游端口的队列。

依据： 9.2.6(56)（第 167 页）。

9.2.6-05　混合来源先服务本地队列

多出连接的 OnDemand 端口同时从最终源和另一个 OnDemand 端口获得输入时，先服务本地队列；本地为空才服务上游队列。

依据： 9.2.6(58)（第 167 页）。

9.2.6-06　多个上游队列按扇入策略选择

多出连接的 OnDemand 端口从多个 OnDemand 队列取得输入时，可依据该端口的扇入优先策略选择服务哪个上游队列。

依据： 9.2.6(58)（第 167 页）。

9.2.6-07　物理传输调整须保持逻辑语义

实现允许实际发送者比逻辑离开时刻更早完成，或实际接收者在逻辑到达后开始；许可条件是观察到的因果顺序与逻辑模型一致，且全部时间属性要求仍得到满足。

依据： 9.2.6(60)（第 167 页）。

9.2.6-08　允许省略必被覆盖的物理传输

周期线程间的数据传输可在能证明该值必被覆盖时省略物理传输；优化可假定传输无错误，但必须保持原有采样结果。

依据： 9.2.6(61)（第 167 页）。

9.2.6-09　允许省略值相同的重复传输

周期线程间的数据传输可在能证明其值与此前已传值相同时省略重复物理传输；优化可假定传输无错误，但必须保持原有采样结果。

依据： 9.2.6(61)（第 167 页）。

9.2.6-10　周期 data 传输错误可被运行时检测

周期线程之间 data 连接的错误或失败传输，须在应用要求的保证程度内可由运行时检测；实现允许补充错误类别及检测机制。

依据： 9.2.6(62)（第 167 页）。

9.2.6-11　Get_Error_Code 报告周期 data 通信错误

周期线程间 data 连接的错误或失败传输被运行时检测后，通过 Get_Error_Code 报告，不由标准统一指定具体错误码数值。

依据：9.2.6(62)（第 167 页）。

9.2.6-12　周期发送失败通知所有接收者

周期发送线程未能执行应完成的传输时，运行时须向该线程所有相连接收者报告发送失败。

依据：9.2.6(62)（第 167 页）。

9.2.6-13　到达传输的数据错误按保证程度检测

任意连接的到达传输中出现数据错误时，运行时须按应用要求的保证程度检测。

依据：9.2.6(62)（第 167 页）。

9.2.6-14　共享 data 源需要监测写入

实现支持以共享 data 为端口连接源时，运行时监测 data 写操作并按连接目标端口语义传送相应输出；实现允许不支持这种源能力。

依据：9.2.6(63)（第 167 页）；9.2.1(16)（第 160–161 页）。

9.3　参数连接

9.3-01　前驱调用结果流向后继调用

调用之间的参数连接按调用序列方向传值：连接源为先行调用的输出，目标为后继调用的输入，不在两次调用之间另行异步发送。

依据：9.3(6)（第 170 页）；5.2(14)–(15)（第 69 页）；9.3(3)（第 170 页）。

9.4　访问连接

9.4-01　不完整访问链以提供特征为终端

尚未展开实际共享子组件的模型中，叶组件的 provides access 可以作为语义访问连接的最终端点；后续展开实际对象时再沿访问链解析到该对象。

依据：9.4(3)（第 170 页）；9.4(L4)（第 172 页）。

9.4-02　双向访问连接不由书写顺序决定共享者

双向语义 access 连接中，被访问子组件与 requires 特征任一端均可写作源或目标；交换文字左右位置不改变所访问的共享实例。

依据：9.4(3)（第 170 页）。

9.4-03　bus 读访问沿总线流向使用者

bus 访问的有效 Access_Right 为读或有向访问连接表示读时，数据从被共享 bus 流向要求访问的组件。

依据： 9.4(9)（第 173 页）；8.7(4)（第 149 页）。

9.4-04　bus 写访问沿使用者流向总线

bus 访问的有效 Access_Right 为写或有向访问连接表示写时，数据从要求访问的组件流向被共享 bus。

依据： 9.4(9)（第 173 页）；8.7(4)（第 149 页）。

9.5　特征组连接

9.5-01　层次 Subset 按同名同向成员连接

跨包含层次的合法 feature group 连接使用 Subset 时，内层输入与输出分别对应外层同名、同方向成员的子集，仅为这些匹配成员展开连接。

依据： 9.5(L3) Subset（第 174 页）；9.5(2)（第 175 页）。

9.5-02　兄弟 Subset 按同名反向成员连接

兄弟组件之间的合法 feature group 连接使用 Subset 时，各方 incoming 成员分别匹配另一方同名 outgoing 成员；未匹配的额外成员不因该组连接获得连接。

依据： 9.5(L4) Subset（第 175 页）；9.5(2)（第 175 页）。

9.5-03　组级方向约束成员数据流

feature group 或其连接指定方向限制时，该限制作用于其代表的成员连接集合，只允许符合该限制的数据流。

依据： 9.5(3)（第 175 页）。

9.5-04　完整特征组展开为成员连接实例

模型已细化到线程且 feature group 成员完整时，将其各成员的语义连接纳入实例模型，沿包含层次进行成员聚合与分解。

依据： 9.5(4)（第 176 页）；9.5(2)（第 175 页）。

9.5-05　不完整特征组保留组间连接实例

线程架构已明确但 feature group 成员尚不完整时，在 feature group 实例之间建立连接实例，不补造未声明成员连接。

依据： 9.5(4)（第 176 页）。

第 10 章　流

10.1　流规格

10.1-01　共享端点的 flow 仍是不同声明

多个 flow specification 涉及相同接口特征时，分别保留各声明的逻辑 flow；后续按各自规格及对应实现展开，不仅因端点相同而合并。

依据：10.1(2)（第 180 页）。

10.2　流实现

10.2-01　flow path 沿声明路径贯通端点

flow path implementation 从规格指定输入出发，按所列连接和子组件走到规格指定输出；允许直接贯通输入输出而没有中间连接，不猜测未声明的内部路线。

依据：10.2(1)（第 184 页）；10.2(3)（第 186 页）。

10.2-02　flow source 以规格输出为终点

flow source implementation 从组件内部或其所声明子组件源出发，沿所列路径终止于对应 flow source specification 指定的输出特征。

依据：10.2(1)（第 184 页）；10.2(3)（第 186 页）。

10.2-03　flow sink 以规格输入为起点

flow sink implementation 从对应 flow sink specification 指定的输入特征进入，沿所列路径终止于组件内部或所声明子组件 sink。

依据：10.2(1)（第 184 页）；10.2(3)（第 186 页）。

10.2-04　同名 flow 实现分别保留路径

同一 flow specification 具有多条同名 flow implementation 时，每条实现保留独立路径，可分别表示冗余路径、组内不同成员路径或模式替代路径；不因同名只留第一条或最后一条。

依据：10.2(N1)（第 185 页）；10.2(2)（第 186 页）。

10.2-05　无 flow 规格时形成未命名贯通路径

flow implementation 仅引用子组件而省略 flow 名，且该子组件没有 flow specification 时，在前一连接目标端与后一连接源端之间假定一个未命名 flow；end-to-end flow 的同类省略采用相同结果。

依据：10.2(4)a（第 186 页）；10.3(3)a（第 189 页）。

10.2-06　省略 flow 名时展开全部既有规格

flow implementation 仅引用子组件而省略 flow 名，且子组件已有 flow specifications 时，为每个规格分别形成 flow；end-to-end flow 的同类省略也分别形成对应路径。

依据： 10.2(4)b（第 186 页）；10.3(3)b（第 189 页）。

10.2-07　调用 flow 进入实际被调实例

线程内部 flow 沿子程序调用参数进入实际被调用实例；经 subprogram access 指向远程子程序时，路径进入该远程实例，不停在访问特征或虚构本地副本。

依据： 10.2(5)（第 186 页）。

10.2-08　flow implementation 按模式生效

flow implementation 的 in_modes_and_transitions 指定该实现适用的配置；未指定 in modes 时，按全部模式适用处理。

依据： 10.2(7)（第 187 页）；12(11)（第 216 页）。

10.3　端到端流

10.3-01　端到端 flow 按段连接拼接

end-to-end flow 引用其他 end-to-end flows 时，将前段末元素通过声明连接接到后段首元素，保留各段内部路径的顺序。

依据： 10.3(4)（第 189 页）。

10.3-02　端到端 flow 递归展开到实际叶节点

end-to-end flow 通过所引用 flow specifications 的 flow implementations 递归展开，形成沿语义连接经过叶组件的路径；完整模型的叶端点为 thread、data、device 或 processor，不完整模型或 flow 实现可停在当前叶 thread group、process 或 system。

依据： 10.3(5)（第 189 页）；10.3(5) 续（第 190 页）。

10.3-03　首段 flow path 确定端到端起点

端到端 flow 的首个子组件 flow specification 为 flow path 时，端到端实例起点取该 flow path 经其实现展开后的首组件。

依据： 10.3(6)（第 190 页）。

10.3-04　末段 flow path 确定端到端终点

端到端 flow 的最后子组件 flow specification 为 flow path 时，端到端实例终点取该 flow path 经其实现展开后的末组件。

依据： 10.3(6)（第 190 页）。

10.3-05　端到端路径按模式配置生效

end-to-end flow 声明带 in_modes_and_transitions 时，该路径仅在所列模式或模式转换中生效。

依据：10.3(7)（第 190 页）。

第 11 章　属性

11.1.1　属性类型

11.1.1-01　枚举成员顺序

枚举类型的成员按声明中的列出顺序排列。

依据： 11.1.1(5)（第 196 页）。

11.1.1-02　单位换算

单位声明中，首单位为基准；后续单位的乘数作用于声明中先前的单位，递归换算得到相同量值。预声明换算包括：Bytes=8 bits；十进制 KByte/MByte/GByte/TByte 每级乘 1000，KiByte/MiByte/GiByte/TiByte 每级乘 1024；时间 ps→ns→us→ms→sec 每级乘 1000，sec→min→hr 每级乘 60；数据速率 Bytesps=8 bitsps，K/M/G 各级乘 1000；处理器速度 MIPS=1000 KIPS、GIPS=1000 MIPS。AADL_Project 可按项目调整并扩展允许的单位，求值依实际声明，不把示例值固定为所有项目的值。

依据： 11.1.1(6)（第 196 页）；11.2(2),(4)（第 200–201 页）；A.8 Data_Rate_Units（第 277 页）；A.8 Size_Units, Time_Units, Processor_Speed_Units（第 278–279 页）。

11.1.1-03　数值与单位组成的值

数值属性类型带 units 时，属性值由数值和单位组成；不带 units 时，属性值只包含数值。

依据： 11.1.1(7)–(8)（第 196 页）；11.4(4)（第 209 页）。

11.1.1-04　区间包含端点

range 属性值表示闭区间，包含给定下界和上界；二者分别是区间的最小值和最大值。

依据： 11.1.1(9)（第 197 页）；A.8 Time_Range、Size_Range（第 277 页）。

11.1.1-05　无 delta 时区间稠密

range 属性值未给出 delta 时，区间按稠密区间解释。

依据： 11.1.1(9)（第 197 页）。

11.1.2　属性定义

11.1.2-01　单值属性的缺省求值

单值属性经过适用关联查找仍未得到值时，采用属性定义的默认表达式；也未声明默认值时，结果为 undefined。

依据： 11.1.2(5)（第 199 页）；11.4(2)（第 209 页）。

11.1.2-02　显式默认列表的求值

已声明默认列表表达式的列表属性，在没有更优先关联时按该表达式求得列表值。

依据： 11.1.2(5)（第 199 页）；11.3(12) 第 1 项（第 205 页）。

11.1.3　属性常量

11.1.3-01　属性常量引用求值

属性表达式中的属性常量引用求值为被引用常量的值，因此可在允许直接写该值的位置用该常量名替代。AADL_Project 中的常量按当前项目声明求值，项目定制常量值后，引用该常量的属性表达式采用定制后的值。

依据： 11.1.3(1)–(2)（第 199–200 页）；11.4(9)（第 210 页）；11.2(2),(L2)（第 200–201 页）；A.8(1)–(2)（第 274 页）。

11.3　属性关联

11.3-01　contained 关联的路径根

contained 属性关联以声明所在模型元素为路径根，沿包含路径定位接受该属性关联的目标元素。

依据： 11.3(N5)（第 203 页）；11.3(8)（第 205 页）。

11.3-02　常量关联保持原值

通过 constant 属性关联确定的值在后续属性值查找与覆盖过程中不能改变；例如类型上 constant 关联的值不会再由该类型的实现、子组件或指向其实例的 contained 关联改写。

依据： 11.3(L9),(16)（第 204,206–207 页）。

11.3-03　声明属性作用于所生实例

分类符、子组件、特征、流、连接、模式和子程序调用等声明上的属性关联，分别用于由该声明生成的实例；实例专属 contained 关联仍可按其优先级确定最终值。

依据： 11.3(2)（第 204 页）；11.3(12) 实例项（第 206 页）。

11.3-04　模态属性值选择

in modes 属性值在列出的任一模式当前激活时适用，并遵循普通属性覆盖规则；同一关联还含无 in modes 值时，该值用于未显式列值的模式，并在组件不活动时适用。模式改变时，相应模态属性值随之选择新当前模式的值。

依据： 11.3(C1),(6)（第 204–205 页）；12(11),(19),(31)（第 216–217 页）。

11.3-05　绑定专属属性值选择

带 in binding 的关联仅在绑定到其中指定的 processor、virtual processor、bus、virtual bus 或 memory 类型时适用。若同一关联列表同时含绑定专属关联及无 in binding 关联，后者用于到 reference processor 的绑定。

依据： 11.3(7)（第 205 页）；13.1(8)（第 222 页）。

11.3-06　多路径属性关联

contained 属性关联列出多个 applies to 路径时，该属性值分别关联到每条路径指向的目标。

依据： 11.3(9)（第 205 页）。

11.3-07　contained 模态属性的模式上下文

contained 关联目标是组件时，其模态值使用目标组件的模式；目标不是组件时，使用包含该目标元素的组件模式。

依据： 11.3(9)（第 205 页）。

11.3-08　分类符类型的属性查找

对组件类型、特征组类型及相应 annex 分类符类型求属性值时，先取其自身 properties 关联；没有时沿 extends 链取最近具有该关联的祖先类型。仍未取得值时进入属性定义默认值处理。

依据： 11.3(12) 第 2 项（第 205 页）。

11.3-09　分类符实现的属性查找

对分类符实现求属性值时，先查自身 properties 关联，再沿 extends 链查最近具有该关联的祖先实现；仍没有时，按类型查找规则取该实现所对应分类符类型的值。上述路径均无值时进入属性定义默认值处理。

依据： 11.3(5)（第 204 页）；11.3(12) 第 3 项（第 205 页）。

11.3-10　无分类符引用元素的属性查找

对模式、连接、流序列等无分类符引用的元素求属性值时，先查本声明，再沿 refinement 链递归查被细化声明；仍无值且属性定义声明 inherit 时，取系统实例包含层次中最近包含组件的非模态值。最后仍无值时进入属性定义默认值处理。

依据： 11.3(12) 第 4 项（第 205 页）。

11.3-11　有分类符引用元素的属性查找

对子组件、特征等有分类符引用的元素求属性值时，先查本声明及 refinement 链，再按被引用分类符的规则查找。仍无值且属性声明 inherit 时，特征组成员先查包含特征组类型及其 extends 链；继而按 inherit 查实例包含层次中最近包含组件的非模态值。全部路径无值时进入属性定义默认值处理。

依据： 11.3(12) 第 5 项（第 205–206 页）。

11.3-12　子程序调用的属性查找

对子程序调用等具有分类符或特征引用的元素求属性值时，先取调用自身关联；没有时，分类符引用按对应分类符规则查，类型中特征引用按该特征规则查。仍无值且属性声明 inherit 时，取实例层次中最近包含组件的非模态值；最后进入属性定义默认值处理。

依据： 11.3(12) 第 6 项（第 206 页）。

11.3-13　实例属性的 contained 优先级

对组件、特征、连接、流、模式等模型元素实例求属性值时，先取实例包含层次中声明位置最高且指向该实例的 contained 关联；没有时，按生成该实例的声明查找；仍无值且属性定义声明 inherit 时，取最近包含组件的非模态值。最后仍无值时进入属性定义默认值处理。

依据： 11.3(12) 第 7 项、(13)、图 21（第 206 页）。

11.3-14　属性关联替换值

=> 属性关联以其求得的新值替换该属性的原值。

依据： 11.3(14)（第 206 页）。

11.3-15　列表关联追加值

合法使用 +=> 时，将求得的值追加到原属性列表末尾；嵌套列表追加到最外层。该运算仅用于列表属性，不用于 contained 关联，也不带 in modes 或 in binding。

依据： 11.3(L5)–(L7)（第 204 页）；11.3(14)（第 206 页）。

11.3-16　列表表达式按序展开

列表属性按出现次序求值各表达式并依序追加其结果；某表达式结果为列表时追加该列表全部元素，结果为 undefined 时将其当作空列表。

依据： 11.3(15)（第 206 页）。

11.3-17　新记录不继承旧字段

对 record 属性建立新关联时，新记录替换旧记录；新记录未列出的字段不从其他属性关联继承。

依据： 11.3(17)（第 207 页）。

11.4　属性表达式

11.4-01　数组 reference 项产生引用列表

reference 项的包含路径使用未限定为单个元素的子组件数组时，结果为所选元素或各元素后续路径目标的引用列表。

依据： 11.4(L6)（第 209 页）；11.4(11)（第 210 页）。

11.4-02　布尔字面量求值

布尔项 true 与 false 分别求值为对应的真值与假值。

依据： 11.4(3)（第 209 页）。

11.4-03　无点无指数的整数字面量类型

无小数点且无指数的数值字面量满足 aadlinteger 类型。

依据： 11.4(4)（第 209 页）；15.4(1)（第 238 页）。

11.4-04　带小数点的实数字面量类型

含小数点的数值字面量满足 aadlreal 类型。

依据： 11.4(4)（第 209 页）；15.4(1)（第 238 页）。

11.4-05　带单位数值的范围比较

数值属性有范围约束时，先按需换算单位，再判断该数值是否位于给定范围内。

依据： 11.4(4)（第 209 页）。

11.4-06　枚举项的值

枚举项求值为该标识符表示的枚举成员。

依据： 11.4(5)（第 209 页）。

11.4-07　区间项求得的数值分量

区间表达式求出下界和上界两个数值；显式给出 delta 时再求出第三个数值 delta。各分量采用该 range 类型规定的数值类型和单位。

依据： 11.4(6)（第 210 页）。

11.4-08　字符串表达式求值

字符串字面量求值为定界双引号之间按序给出的字符；其中成对双引号变为一个双引号，其余字符保持原样；没有字符串元素时结果为空字符串。

依据： 11.4(7)（第 210 页）；15.5(3)–(4)（第 239 页）。

11.4-09　属性引用在目标上下文求值

属性表达式引用另一个属性时，在当前正在确定属性值的模型元素上下文求被引用属性，而不是在默认值或表达式原声明位置求值；例如 Deadline 的默认表达式 Period 取当前线程子组件的 Period。被引用属性 undefined 时该项也为 undefined。

依据： 11.4(8)（第 210 页）。

11.4-10　分类符表达式求值

classifier 表达式求值为对指定分类符的引用。

依据： 11.4(10)（第 210 页）。

11.4-11　单目标 reference 项求值

reference 项的包含路径限定单个目标时，求值为对该模型元素的引用；目标可为组件、连接、模式、特征或适用 annex 模型元素。

依据： 11.4(11)（第 210 页）。

11.4-12　记录显式字段按名称求值

记录表达式中显式列出的每个字段分别求值，以字段名对应记录中的相应字段值。

依据： 11.4(12)（第 210 页）。

11.4-13　compute 的调用求值与受限缓存

每次访问 compute 属性值时调用指定用户函数，并以属性关联及关联模型元素为参数取得返回值；函数可读取同一或其他元素的属性。实现可施加限制后仅计算一次并缓存，例如依赖属性由关联声明确定且不会程序化改变时，可在实例化时求值；未满足所采用限制时，不能一般地以首次缓存代替后续调用。

依据： 11.4(13)（第 210 页）；11.4(18)（第 211 页）。

11.4-14　compute 不改变被求值模型

用于计算属性值的用户函数预期没有副作用；计算属性值本身不修改模型状态。

依据： 11.4(13)（第 210 页）。

11.4-15　compute 在有限时间完成

用于计算属性值的用户函数假定在有限时间内完成计算。

依据： 11.4(13)（第 210 页）。

11.4-16　compute 返回相容类型值

用于计算属性值的用户函数预期返回与该属性类型相容的值。

依据： 11.4(13)（第 210 页）。

11.4-17　缺失所需属性的处理选择

属性既无关联表达式也无默认值、且处理活动需要其值时，处理方法可按属性选择拒绝规格，或采用另行构造且已告知开发者的默认表达式。

依据： 11.4(15)（第 210 页）。

11.4-18　无关属性缺值不得阻止处理

具体开发活动不需要某属性值时，该活动的处理方法必须接受未关联该值的规格。

依据： 11.4(15)（第 210 页）。

11.4-19　程序化修改记录字段

处理方法可通过程序读取和修改记录属性值的字段。

依据： 11.4(17)（第 211 页）。

第 12 章　模式

12-01　self 事件可以请求模式转换

组件自身产生的 self 事件可作为模式转换触发源；仅触发当前模式中命名该内部事件的出转换。

依据： 12(2)（第 213 页）；12(16)（第 216 页）；12 NOTE after (26)（第 217 页）。

12-02　连接细化的有效模式集合

连接 refined to 声明带 in modes 时，以新集合替换原连接的模式成员集合；细化声明没有 in modes 时，连接在全部模式活动。

依据： 12(N6)（第 214 页）；12(14)（第 216 页）。

12-03　父模式映射为所需子模式

子组件使用 requires modes 时，其 in modes 中的 => 把父模式映射到指定子模式；未写 => 时按同名映射。多个父模式允许映射到同一子模式，组件的模式相关声明按映射所得子模式解释。

依据： 12(L4)（第 215 页）；12(12)（第 216 页）。

12-04　架构模式决定分派资格

线程组、进程或系统的当前模式决定哪些线程可响应分派；当前模式中的线程可以等待分派，其他线程等待模式或处于 halted。

依据： 12(5)（第 215 页）。

12-05　平台模式限制可访问资源

系统或执行平台组件使用带 in modes 的子组件时，软件只能访问当前模式中的执行平台组件；例如活动应用组件只能绑定到同模式活动的处理器和内存。

依据： 12(7)（第 215 页）。

12-06　组件当前模式唯一

声明了模式的组件任一时刻恰有一个当前模式。

依据： 12(9)（第 215 页）。

12-07　初始当前模式

有模式的组件最初以声明为 initial 的模式作为当前模式。

依据： 12(9)（第 215 页）。

12-08　重新激活的当前模式

有自身模式的组件因外层模式而停用后重新激活时，Resumption_Policy=restart 使其进入 initial 模式，resume 使其恢复上次停用时的当前模式。

依据： 12(10)（第 215 页）；A.2 Resumption_Policy（第 251 页）。

12-09　多个模式触发源默认作析取

同一模式转换列出多个触发端口时，默认任一所列端口产生触发事件即满足触发条件；附录可以定义其他组合条件。

依据： 12(16)（第 216 页）；12 NOTE after (26)（第 217 页）。

12-10　同时转换事件的选择

多个逻辑同时的触发事件命中当前模式不同转换时，若所有被命名端口都有 Urgency，最高紧急度转换优先，同最高值时非确定选择；没有满足这一属性条件时，到达次序由实现决定。同样的优先选择适用于逻辑同时发起的多个 SOM 转换。

依据： 12(18)（第 216 页）；13.6(3)（第 228 页）。

12-11　分派时的模式选择本次行为

每次分派以分派时当前内部模式选择本次代码序列、入口、调用序列及相应执行时间。

依据： 12(20)（第 216 页）。

12-12　平台内部切模保持外部活动结构

执行平台组件内部模式转换不改变系统活动线程、处理器、设备、总线、内存及该组件外部活动连接的集合。

依据： 12(22)（第 217 页）。

12-13　线程内部模式不改变外部活动结构

线程内部模式转换选择本线程行为，不改变外部活动线程或平台组件集合，也不改变该线程外部的活动连接集合。

依据： 12(22)（第 217 页）。

12-14　持续活动线程保持执行

新旧模式都活动的线程继续正常分派和执行；其中 Background 线程继续 execute 状态，不因这次切换重新启动计算。

依据： 12(23)–(24)（第 217 页）。

12-15　核心模式转换不判断消息数据值

核心模式转换声明不按 event data 的内容选择转换；需要依数据决定转换时，由执行逻辑解释数据并发出相应触发事件，附录另有定义的条件按该附录处理。

依据： 12 NOTE after (26)（第 217 页）。

12-16　模式切换的队列清空和保持

实际模式切换时，可继承属性 Active_Thread_Queue_Handling_Protocol 决定线程 event 及 event data 端口队列的处理：flush 清空队列，hold 把停用线程队列内容保持到线程再次激活；默认 flush。实现允许为 aperiodic 和 sporadic 线程提供这种跨切换保持能力，选用 hold 时按保持分支处理。

依据： 12(29)（第 217 页）；13.6(20)–(21)（第 229–230 页）；A.2 Active_Thread_Queue_Handling_Protocol（第 252 页）。

第 13 章　系统运行语义

13.1　系统实例

13.1-01　模式只选择静态配置

系统实例的包含结构和互连拓扑静态已知；运行时模式转换在预先确定的替代配置之间选择活动配置，不创建声明之外的新组件或新连接。

依据： 13.1(5)（第 222 页）。

13.1-02　处理器内核地址空间的内容

每个处理器具有自己的内核地址空间，包含绑定到该处理器的处理器软件和设备驱动软件的二进制镜像。

依据： 13.1(7)（第 222 页）。

13.2　应用软件的执行平台绑定

13.2-01　动态装载允许改变进程绑定

采用动态进程装载时，实际进程绑定允许在运行期改变；具体实现仍可把某类绑定固定在开发时、构建时或指定阶段，或允许其动态变化。

依据： 13.2(C2)（第 223 页）；13.2(4)（第 224 页）。

13.2-02　多处理器绑定不并行复制线程

线程绑定多个共享 ready queue 的处理器时，调度器可在该绑定集合内选择执行处理器；线程任一时刻只在其中一个处理器执行。

依据： 13.2(C2),(C5)（第 223 页）；A.1 Actual_Processor_Binding（第 242 页）。

13.2-03　进程各执行者均可访问其内存

进程全部软件组件绑定的每块内存，都对该进程任一线程所绑定的每个处理器可访问。

依据： 13.2(C7)（第 223 页）；13.3(5)（第 226 页）。

13.2-04　共享数据对全部共享者可达

共享数据组件的已绑定内存，对所有共享线程的执行处理器可访问；共享跨进程时仍满足这一可达性。

依据： 13.2(C8)（第 223 页）；13.3(6)（第 226 页）。

13.2-05　处理器候选集合展开

Allowed_Processor_Binding 直接列出的 processor 或 virtual processor 进入候选集合，system 引用展开为其包含的处理器；单一处理器候选确定精确目标。该可继承属性未被局部覆盖时采用容器值；无关联值时，全部已声明处理器均为候选。实际选择仍须满足分类器限制及资源可达性。

依据： 13.2(N1)（第 222 页）；13.2(C11)–(C12)（第 223 页）；A.1 Allowed_Processor_Binding（第 242 页）。

13.2-06　内存候选集合展开

Allowed_Memory_Binding 直接列出的 memory 进入候选集合，processor 引用展开为其中包含的内存，system 引用展开为其包含的内存及所含处理器内存；单一内存候选确定精确目标。该可继承属性未被局部覆盖时采用容器值；无关联值时，全部已声明内存均为候选。实际选择仍须满足分类器限制及资源可达性。

依据： 13.2(N2)（第 222 页）；13.2(C13)（第 223 页）；A.1 Allowed_Memory_Binding（第 243 页）。

13.2-07　运行内存容量违规产生错误

实现可静态或动态检查内存约束；若运行时检测到已绑定二进制镜像及线程、进程的内存需求超过目标内存容量，则产生应用可以选择恢复的错误。

依据： 13.2(C14)（第 223 页）。

13.2-08　运行时间需求违规产生错误

实现可静态或动态检查处理器时间约束；若运行时检测到处理器上的线程执行需求超过满足全部线程时间要求所允许的可调度周期，则产生应用可以选择恢复的错误。

依据： 13.2(C14)（第 223 页）。

13.2-09　跨进程共享内存的地址可不同

多个进程共享一个组件时，其物理内存出现在各进程虚拟地址空间中；不同进程的虚拟地址或不同处理器采用的物理地址允许不同，地址不同不改变所共享组件的身份。

依据： 13.2 NOTE after (5)（第 224 页）。

13.3　系统启动

13.3-01　系统启动请求进入 starting

system offline 收到 start(system) 后进入 system starting。

依据： 13.3(1)（第 226 页）；13.3 Figure 22（第 227 页）。

13.3-02　系统启动发起平台初始化

系统由 offline 启动时，start(system) 发起执行平台组件的初始化。

依据： 13.3(1)（第 226 页）。

13.3-03　平台软件初始化前装载内核镜像

系统启动处理器时，先把处理器内核地址空间的二进制镜像加载到内存，再执行平台软件初始化。

依据： 13.3(1)（第 226 页）。

13.3-04　预装内核镜像允许零装载时间

处理器内核镜像可预装于 PROM 或 flash 等内存时，系统启动中的该镜像装载动作允许耗时为零。

依据： 13.3(1)（第 226 页）。

13.3-05　静态装载先完成全部映像

采用静态进程装载时，线程初始化开始前须装完所有进程二进制映像。

依据： 13.3(3)（第 226 页）；13.7(22)（第 232 页）。

13.3-06　动态装载先完成当前模式映像

采用动态进程装载时，应用执行前须装完所有包含当前模式活动线程的进程映像；实现允许仅支持静态装载。

依据： 13.3(3)（第 226 页）；13.7(16)（第 232 页）。

13.3-07　系统初始化时间上界

各阶段参与组件集合均非空时，系统初始化时间上界为五项之和：处理器及其他硬件的最大 Startup_Deadline、虚拟处理器的最大 Startup_Deadline、进程的最大 Load_Deadline、进程的最大 Process_Startup_Deadline、线程的最大 Initialize_Deadline。

依据： 13.3(4)（第 226 页）。

13.3-08　初始化须赶上下个超周期边界

系统启动中的线程初始化须在初始模式的下一个 hyperperiod 边界前完成。

依据： 13.3(7)（第 226 页）。

13.3-09　初始模式的全局就绪条件

全体线程完成初始化后，初始模式中的线程进入 await dispatch 并参与系统执行；已经装载但不在初始模式中的线程进入 suspend awaiting mode。

依据： 13.3(7)（第 226 页）。

13.3-10　首次分派可取得端口初值

接收线程首次分派时，已连接源 out 或 in out 端口的初始值在接收线程的 in 或 in out 端口可用。

依据： 13.3(7)（第 226 页）。

13.3-11　线程初始化的依赖范围

默认各线程独立初始化，只要求所属进程先初始化；模型通过初始化模式明确了线程执行依赖时，按该依赖协调初始化。

依据： 13.3(8)（第 226 页）。

13.3-12　系统启动将 ST 清零

system offline 收到 start(system) 并转入 system starting 时，系统启动计时 ST 清零。

依据： 13.3 Figure 22（第 227 页）。

13.3-13　系统 starting 的期限约束

系统保持在 starting 状态时，启动计时满足 ST≤startup_deadline。

依据： 13.3 Figure 22（第 227 页）。

13.3-14　系统启动完成进入 operational

系统处于 starting 时，started(system) 使其进入 operational；该完成边要求 ST≤startup_deadline。

依据： 13.3 Figure 22（第 227 页）。

13.3-15　运行系统开始受控停止

operational 中的系统收到 stop(system) 后进入 system stopping。

依据： 13.3 Figure 22（第 227 页）。

13.3-16　系统停止完成回到 offline

系统处于 stopping 时，stopped(system) 使其返回 system offline。

依据： 13.3 Figure 22（第 227 页）。

13.3-17　启动或运行系统中止回到 offline

系统在 starting 或 operational 状态收到 abort(system) 后，直接返回 system offline。

依据： 13.3 Figure 22（第 227 页）。

13.4　正常系统运行

13.4-01　同一事件沿扇出连接同时到达

在同步域内，一个已发出的事件沿所有以其发送端为最终源的语义连接，在各最终目的端逻辑同时到达；异步系统跨域只有实现提供协调协议时才具有其约定时间粒度内的同时性保证，否则跨域到达彼此独立。

依据： 13.4(4)–(5)（第 227 页）。

13.4-02　全局时钟粒度判定逻辑同时

同步系统中，两个事件若发生在全局同步参考时钟的同一粒度内，就视为逻辑同时产生；异步系统中此判据限于同一同步域，除非实现提供跨域协调保证。

依据： 13.4(4)–(5)（第 227 页）。

13.4-03　同时事件的到达顺序由实现决定

多个逻辑同时产生的事件到达同一端口，或同一或不同组件当前模式的不同出边时，其到达顺序由实现决定。

依据： 13.4(4)（第 227 页）。

13.4-04　跨同步域分派可独立

异步系统只在每个同步域内保证周期分派对齐；跨域没有提供有界时间粒度协调的协议时，各域可独立分派。

依据： 13.4(5)（第 227 页）。

13.5　系统运行模式

13.5-01　候选 SOM 由组件模式组合产生

一个系统操作模式 SOM 为系统各组件各取一个模式的组合；候选 SOM 集合是各组件模式集合的笛卡尔积。

依据： 13.5(1)（第 227 页）。

13.5-02　初始 SOM 取各组件初始模式

系统的初始 SOM 由每个组件的 initial 模式组成。

依据： 13.5(1)（第 227 页）。

13.5-03　不可达模式组合可从候选 SOM 删除

候选 SOM 可删去不可达组合：组件在某祖先模式中不活动时可排除相应组合；组件继承包含组件的模式时只需考虑包含组件的模式组合；结合模式转移条件的可达性分析还可进一步减少候选集合。

依据： 13.5(2)（第 227 页）。

13.6　系统运行模式转换

13.6-01　同一事件的组件切换合为一个 SOM 请求

同一个已发送 event 或 event data 可触发一个或多个组件的模式切换；在同步系统中，这些触发逻辑同时发生，所产生的组件模式转移请求作为一次 SOM 转移处理。

依据： 13.6(1)（第 228 页）。

13.6-02　不同同时事件分别发起 SOM 请求

多个不同事件即使逻辑同时产生，包括一次 Send_Output 调用向多个端口发送的情形，只要分别语义连接到组件当前模式的出边，就各自作为独立的 SOM 转移发起。

依据： 13.6(2)（第 228 页）。

13.6-03　线程内部转换在下一分派生效

线程及其子程序内部的模式转换在下一次线程分派时生效，不以该内部转换立即重配当前正在进行的分派。

依据： 13.6(4)（第 228 页）。

13.6-04　平台内部模式立即转换

processor、device、bus 或 memory 内部模式的有效自身事件或外部事件触发转换时，该内部模式转换立即发生。

依据： 13.6(5)（第 228 页）。

13.6-05　Emergency 立即进入实际模式转移

Mode_Transition_Response 为 Emergency 时，系统在零时间内进入 mode_transition_in_progress，立即开始实际 SOM 转移。

依据： 13.6(6)（第 228 页）。

13.6-06　Emergency 中止待停用的执行线程

Emergency 模式转移中，所有仍处于 performing 状态且需要停用的线程被 abort。

依据： 13.6(6)（第 228 页）。

13.6-07　Planned 等待旧模式关键周期对齐

Mode_Transition_Response 不是 Emergency 时采用 Planned 响应；该属性缺省也为 planned。系统继续按旧 SOM 执行，直到旧 SOM 中关键周期线程及设备的分派在其 hyperperiod 处对齐，才进入 mode_transition_in_progress；关键集合为空时立即进入。关键组件满足 Synchronized_Component=true，该属性可继承且缺省 true。

依据： 12(17)（第 216 页）；13.6(7), Figure 23（第 228,230 页）；A.2 Mode_Transition_Response（第 251 页）；A.2 Synchronized_Component（第 252 页）。

13.6-08　等待模式转移时忽略非更高优先级请求

已经收到 SOM 请求而尚未进入 mode_transition_in_progress 时，能从当前 SOM 发起转移、但 Urgency 等于或低于待处理请求的新触发事件被忽略。

依据： 13.6(8)（第 228 页）。

13.6-09　等待模式转移时更高优先级请求替换候选

已经收到 SOM 请求而尚未进入 mode_transition_in_progress 时，具有更高 Urgency 的新触发事件取代当前候选；实际进入转移状态时以替换后的事件确定 SOM 转移。

依据： 13.6(8)（第 228 页）。

13.6-10　实际 SOM 转移耗时非零

运行期间从一个 SOM 实际转移到另一个 SOM 需要非零时间区间。

依据： 13.6(9)（第 228 页）。

13.6-11　模式转移内部忽略后续触发

处于实际 SOM 转移期间时，所有模式转移触发事件均就模式转移而言被忽略；此范围不含转移开始和结束的两个时刻。

依据： 13.6(9)（第 228 页）。

13.6-12　模式转移完成后按持续关键线程对齐退出

持续执行的关键组件集合非空时，SOM 转移在全部必要的线程及连接停用、激活动作完成后，于全局参考时钟 ST 的后续 hyperperiod 对齐边界退出 mode_transition_in_progress；该集合为新旧 SOM 均活动且 Synchronized_Component=true 的周期线程，该属性可继承且缺省 true。

依据： 13.6(10), Figure 23（第 228–230 页）；A.2 Synchronized_Component（第 252 页）。

13.6-13　模式转移结束后新模式线程开始执行

SOM 转移结束并进入新的 current_system_operation_mode 时，新模式中的活动线程开始执行。

依据： 13.6(10)（第 228–229 页）。

13.6-14　转移开始时禁用旧模式独有连接

进入 mode_transition_in_progress 的瞬间，属于旧 SOM 而不属于新 SOM 的连接被禁用。

依据： 13.6(12)（第 229 页）。

13.6-15　转移结束时启用新模式独有连接

离开 mode_transition_in_progress 的瞬间，属于新 SOM 而不属于旧 SOM 的连接被启用。

依据： 13.6(14)（第 229 页）。

13.6-16　新模式启用连接时传递源数据

退出 mode_transition_in_progress 并启用新模式连接时，源线程 out 数据值传入新启用线程的 in 数据端口；若该目的 in 端口也是本次模式转移专用连接的目的端，则不进行这次普通启用传输。

依据： 13.6(14)（第 229 页）。

13.6-17　持续线程在转移期只用两模式共有连接

在 mode_transition_in_progress 期间，旧 SOM 与新 SOM 均活动的持续线程只能使用两个 SOM 中均活动的连接。

依据： 13.6(15)（第 229 页）。

13.6-18　转换开始时同步发出停用请求

系统进入 mode_transition_in_progress 时，对旧模式独有线程同步发出 thread exit(Mode)；普通线程执行停用入口，zombie 线程另按其处理协议。

依据： 13.6(16)（第 229 页）；13.6(23)（第 230 页）。

13.6-19　转换开始时同步发出激活请求

系统进入 mode_transition_in_progress 时，对新模式独有线程同步发出 thread enter(Mode)；按普通激活路径的线程由此执行激活入口。

依据： 13.6(17)（第 229 页）；13.6(23)（第 230 页）。

13.6-20　激活不必等待全部停用结束

模式转换中，不要求所有停用入口均执行完后才开始任何激活入口；两类入口的执行允许重叠。

依据： 13.6(18)（第 229 页）。

13.6-21　激活与停用阶段的时间上界

模式转换中，线程激活和停用阶段的最大执行时间按相应入口 deadline 的最大值确定，不按所有入口 deadline 之和计算。

依据： 13.6(18)（第 229 页）。

13.6-22　Abort 停用协议停止当前执行

对旧模式活动而新模式不活动的 aperiodic、sporadic、timed、hybrid 线程，以及同样由活动变不活动且分派未在 hyperperiod 处对齐的 periodic 线程，若可继承的 Active_Thread_Handling_Protocol 为 abort（缺省值），实际模式切换时停止其当前执行。

依据： 13.6(11),(19)–(20)（第 229 页）；A.2 Active_Thread_Handling_Protocol（第 251 页）。

13.6-23　Abort 停用协议调用 recover

对旧模式活动而新模式不活动的 aperiodic、sporadic、timed、hybrid 线程，以及同样由活动变不活动且分派未在 hyperperiod 处对齐的 periodic 线程，若可继承的 Active_Thread_Handling_Protocol 为 abort（缺省值），实际模式切换停止当前执行后调用 recover 入口，并指示此次停止是为停用而进行；recover 入口允许恢复一致状态及释放所持资源。

依据： 13.6(11),(19)–(20)（第 229 页）；A.2 Active_Thread_Handling_Protocol（第 251 页）。

13.6-24　停用 recover 完成后等待模式

对旧模式活动而新模式不活动的 aperiodic、sporadic、timed、hybrid 线程，以及同样由活动变不活动且分派未在 hyperperiod 处对齐的 periodic 线程，若可继承的 Active_Thread_Handling_Protocol 为 abort（缺省值），为该次停用执行的 recover 入口完成后，线程进入 suspended awaiting mode。

依据： 13.6(11),(19)–(20)（第 229 页）；A.2 Active_Thread_Handling_Protocol（第 251 页）。

13.6-25 服务线程停用时中止远程调用者当前分派

对旧模式活动而新模式不活动的 aperiodic、sporadic、timed、hybrid 线程，以及同样由活动变不活动且分派未在 hyperperiod 处对齐的 periodic 线程采用默认 abort 停用时，若该线程正在执行远程调用的子程序，则正在进行或排队的远程调用所对应的调用线程，其当前分派执行也被 abort。

依据： 13.6(11),(20)（第 229 页）。

13.6-26 停用恢复期限计入模式转移时长

对旧模式活动而新模式不活动的 aperiodic、sporadic、timed、hybrid 线程，以及同样由活动变不活动且分派未在 hyperperiod 处对齐的 periodic 线程采用 abort 并执行 recover 入口时，recover 入口的 deadline 计入 mode_transition_in_progress 持续时间的确定。

依据： 13.6(11),(20)（第 229–230 页）。

13.6-27 Suspend 停用协议挂起执行

实现若提供并为旧模式活动而新模式不活动的 aperiodic、sporadic、timed、hybrid 线程，以及同样由活动变不活动且分派未在 hyperperiod 处对齐的 periodic 线程选择 Suspend 停用协议，实际模式切换时挂起其执行。Suspend 属于项目特定可选协议，实现可以只支持协议集合的子集；该协议只在线程挂起时未持有共享资源锁的情况下安全。

依据： 13.6(11),(19)（第 229 页）；13.6(21) Suspend（第 230 页）；12(30)（第 217 页）；A.8 Supported_Active_Thread_Handling_Protocols（第 274 页）。

13.6-28 Suspend 在线程再次激活时恢复

实现若提供并为旧模式活动而新模式不活动的 aperiodic、sporadic、timed、hybrid 线程，以及同样由活动变不活动且分派未在 hyperperiod 处对齐的 periodic 线程选择 Suspend 停用协议，已挂起的执行在线程下次成为活动模式成员时恢复。Suspend 属于项目特定可选协议，实现可以只支持协议集合的子集；该协议只在线程挂起时未持有共享资源锁的情况下安全。

依据： 13.6(11),(19)（第 229 页）；13.6(21) Suspend（第 230 页）；12(30)（第 217 页）；A.8 Supported_Active_Thread_Handling_Protocols（第 274 页）。

13.6-29 Complete_one 允许完成当前分派

实现若提供并为旧模式活动而新模式不活动的 aperiodic、sporadic、timed、hybrid 线程，以及同样由活动变不活动且分派未在 hyperperiod 处对齐的 periodic 线程选择 Complete_one 停用协议，实际模式切换时允许其继续完成当前分派。Complete_one 为项目特定可选协议，实现可以只支持协议集合的子集。

依据： 13.6(11),(19)（第 229 页）；13.6(21) Complete_one（第 230 页）；12(30)（第 217 页）；A.8 Supported_Active_Thread_Handling_Protocols（第 274 页）。

13.6-30 Complete_one 完成分派后执行停用

实现若提供并为旧模式活动而新模式不活动的 aperiodic、sporadic、timed、hybrid 线程，以及同样由活动变不活动且分派未在 hyperperiod 处对齐的 periodic 线程选择 Complete_one 停用协议，当前分派完成后调用 deactivation 入口。Complete_one 为项目特定可选协议，实现可以只支持协议集合的子集。

依据： 13.6(11),(19)（第 229 页）；13.6(21) Complete_one（第 230 页）；12(30)（第 217 页）；A.8 Supported_ Active_Thread_Handling_Protocols（第 274 页）。

13.6-31 Complete_all 处理切换时队列

实现若提供并为旧模式活动而新模式不活动的 aperiodic、sporadic、timed、hybrid 线程，以及同样由活动变不活动且分派未在 hyperperiod 处对齐的 periodic 线程选择 Complete_all 停用协议，允许其处理完进入实际模式切换状态时队列中的全部事件或事件数据。Complete_all 为项目特定可选协议，实现可以只支持协议集合的子集。

依据： 13.6(11),(19)（第 229 页）；13.6(21) Complete_all（第 230 页）；12(30)（第 217 页）；A.8 Supported_ Active_Thread_Handling_Protocols（第 274 页）。

13.6-32 Complete_all 完成队列处理后执行停用

实现若提供并为旧模式活动而新模式不活动的 aperiodic、sporadic、timed、hybrid 线程，以及同样由活动变不活动且分派未在 hyperperiod 处对齐的 periodic 线程选择 Complete_all 停用协议，进入实际模式切换状态时队列中的全部事件或事件数据处理完成后，执行 deactivation 入口。Complete_all 为项目特定可选协议，实现可以只支持协议集合的子集。

依据： 13.6(11),(19)（第 229 页）；13.6(21) Complete_all（第 230 页）；12(30)（第 217 页）；A.8 Supported_ Active_Thread_Handling_Protocols（第 274 页）。

13.6-33 完成型停用协议继续竞争执行资源

对旧模式活动而新模式不活动的 aperiodic、sporadic、timed、hybrid 线程，以及同样由活动变不活动且分派未在 hyperperiod 处对齐的 periodic 线程选用实现提供的 Complete_one 或 Complete_all 协议时，其获准继续的执行仍与其他工作竞争执行平台资源。

依据： 13.6(11)（第 229 页）；13.6(21) Complete_one/Complete_all（第 230 页）。

13.6-34 完成型停用协议保留输出至重新激活

对旧模式活动而新模式不活动的 aperiodic、sporadic、timed、hybrid 线程，以及同样由活动变不活动且分派未在 hyperperiod 处对齐的 periodic 线程选用实现提供的 Complete_one 或 Complete_all 协议时，输出被保留至线程重新激活，再按届时的连接传送。

依据： 13.6(11)（第 229 页）；13.6(21) Complete_one/Complete_all（第 230 页）。

13.6-35 回到当前 SOM 状态后重新接受切换请求

系统实例在 SOM 转移超周期的下一个对齐时刻进入 current_system_operation_mode 后，重新响应新的 SOM 转移请求。

依据： 13.6(22), Figure 23（第 230 页）。

13.6-36 模式转移在各同步域内协调

系统操作模式转移涉及的线程与连接激活、停用必须在每个同步域内协调；同步系统的协调范围就是其单一同步域。

依据： 13.6(24)（第 230 页）。

13.6-37　异步系统可提供跨域有界协调

异步系统实现可提供协调协议，使 SOM 转移在有界时间内跨同步域协调；未提供该协议时，标准只要求各同步域内的协调，不由此取得跨域原子切换保证。

依据： 13.6(24)（第 230 页）。

13.7　系统范围的故障处理、关闭与重启

13.7-01　读取当前系统模式

Current_System_Mode 返回与当前活动 SOM 对应的标识值；标识表示由实现决定，不自行规定编码。

依据： 13.7(3)（第 231 页）；A.9(19),(21)（第 282–283 页）。

13.7-02　请求下一个系统模式

Set_System_Mode 以标识指定运行系统下一次必须切换到的 SOM。

依据： 13.7(4)（第 231 页）；A.9(20),(22)（第 283 页）。

13.7-03　虚拟处理器停止递归到下层虚拟处理器

调用 Stop_Virtual_Processor 时，为所有绑定到该虚拟处理器的虚拟处理器发起 Stop_Virtual_Processor。

依据： 13.7(8)（第 231 页）。

13.7-04　虚拟处理器停止传播到绑定进程

调用 Stop_Virtual_Processor 时，对所有绑定到该虚拟处理器的进程发起 Stop_Process。

依据： 13.7(8)（第 231 页）。

13.7-05　虚拟处理器中止传播到下层虚拟处理器

调用 Abort_Virtual_Processor 时，中止所有绑定到该虚拟处理器的虚拟处理器。

依据： 13.7(9)（第 231 页）。

13.7-06　虚拟处理器中止传播到绑定进程

调用 Abort_Virtual_Processor 时，中止所有绑定到该虚拟处理器的进程。

依据： 13.7(9)（第 231 页）。

13.7-07　处理中止传播至绑定进程

以处理器 ID 调用 Abort_Processor 时，所有绑定到该处理器的进程均被中止。

依据： 13.7(11)（第 231 页）。

13.7-08　Stop_System 级联停止各处理器

Stop_System 对系统中的每个处理器发起 Stop_Processor。

依据： 13.7(12)（第 231 页）；A.9(30)（第 283 页）。

13.7-09　Abort_System 中止全部处理器

因异常调用 Abort_System 时，系统中的全部处理器被 abort。

依据： 13.7(13)（第 231 页）；A.9(31)（第 283 页）。

13.7-10　进程映像允许独立组织

实现可以将进程的源命名空间、虚拟地址空间和可装载二进制映像分开组织；一个进程不必只对应一个物理映像。

依据： 13.7(17)–(19)（第 232 页）。

13.7-11　允许省略无人使用的内核软件

绑定到某处理器的全部线程均不需要访问或执行某处理器软件、设备驱动或总线协议软件时，实现可省略在该处理器内核地址空间装载这部分软件。

依据： 13.7(21)（第 232 页）。

13.7-12　替代流程按模式重新装载进程

实现可对指定进程子集采用替代普通线程激活动作及进程装载动作的流程；采用该流程时，每次系统进入使该进程任一线程活动的模式，都重新装载该进程。选中进程的全部线程须一致采用该替代语义，不得在同一进程内混用。

依据： 13.7(24)（第 232 页）。

13.7-13　替代流程按模式重新初始化进程

实现可对指定进程子集采用替代普通线程激活动作及进程装载动作的流程；采用该流程时，每次系统进入使该进程任一线程活动的模式，都重新初始化该进程。选中进程的全部线程须一致采用该替代语义，不得在同一进程内混用。

依据： 13.7(24)（第 232 页）。

第 14 章　分层系统的建模

14-01　平台实例可由实现模式替换

建立实例模型时，processor、virtual processor 或 device 可采用 Implemented_As 关联的 system implementation 替换原平台抽象实例；采用这一替换属于许可，不是所有实例化都必须展开的步骤。

依据： 14(7)（第 233 页）。

14-02　替换结构保持平台外部接口

采用 Implemented_As 系统实现替换 processor、virtual processor 或 device 时，替换系统实现原平台元素类型规定的接口，外部交互仍经该接口进行。

依据： 14(7)（第 233 页）。

14-03　总线实现重定向原连接源段

采用 bus 或 virtual bus 的系统实现重定向已绑定连接时，原连接源端改连到该系统实现中的源特征。

依据： 14(8)（第 233 页）。

14-04　总线实现重定向原连接目的段

采用 bus 或 virtual bus 的系统实现重定向已绑定连接时，该系统实现的预定义目的特征连接到原连接目的端。

依据： 14(8)（第 233 页）。

14-05　总线 API 映射采用发送服务

采用 API 式服务结构替代总线连接映射时，通信的发送操作可以映射为 Send_Output 服务调用。

依据： 14(8)（第 233 页）；8.3.5(41)（第 135 页）。

14-06　总线 API 映射采用接收服务

采用 API 式服务结构替代总线连接映射时，通信的接收操作可以映射为 Receive_Input 服务调用。

依据： 14(8)（第 233 页）；8.3.5(44)（第 135 页）。

14-07　内存实现映射读取

采用 memory 的系统实现时，对绑定数据的读取可映射为读取该系统实现内部数据组件，或映射为对该系统 subprogram access 特征的 retrieve 调用。

依据： 14(9)（第 234 页）。

14-08　内存实现映射写入

采用 memory 的系统实现时，对绑定数据的写入可映射为写入该系统实现内部数据组件，或映射为对该系统 subprogram access 特征的 store 调用。

依据： 14(9)（第 234 页）。

15.4.1　十进制字面量

15.4.1-01　十进制字面量的数值

十进制字面量的值为无指数部分的十进制数值乘以 10 的指数次幂；没有指数时直接取该十进制数值。数字中的下划线及指数 E 的大小写不改变结果。

依据： 15.4.1(1)–(3)（第 238 页）。

15.4.2　基数字面量

15.4.2-01　基数字面量的数值

合法基数字面量按所给基数解释数码，A 至 F 及其小写表示 10 至 15；带指数时，再乘以该基数的指数次幂。基数和指数自身以十进制表示，数码间下划线和字母数码大小写不改变数值。

依据： 15.4.2(L1)（第 238 页）；15.4.2(2)–(3)（第 239 页）。

附录 A　预声明属性集与运行时服务

A-01　预声明属性集自动存在

标准预声明属性集及 AADL_Project 属于每个 AADL 规格，无须用户另行导入其定义；标准预声明属性集的定义固定。

依据：附录 A(1)–(2)（第 241 页）。

A-02　项目类型和常量采用项目定制值

AADL_Project 中的枚举类型和属性常量允许按项目或站点定制；引用这些定义的属性按该项目实际采用的定义和值解释，不使用某个全局固定项目值。

依据：附录 A(2)（第 241 页）。

A-03　abstract 可关联全部属性

所有预声明及用户定义属性均适用于 abstract 组件，即使属性定义的 applies to 未列出 abstract 类别。

依据：附录 A NOTE（第 241 页）。

A.1　预声明部署属性

A.1-01　处理器候选按分类器筛选

存在有效 Allowed_Processor_Binding_Class 值时，从 Allowed_Processor_Binding 候选中只保留满足所列分类器的资源；没有该分类器约束时，不作这一筛减。

依据：A.1 Allowed_Processor_Binding_Class（第 241 页）；13.2(C11)（第 223 页）。

A.1-02　设备允许绑定约束驱动线程

device 上的 Allowed_Processor_Binding 给出其设备驱动线程可以执行的平台候选。

依据：A.1 Allowed_Processor_Binding（第 242 页）。

A.1-03　线程实际执行平台由绑定属性确定

线程的 Actual_Processor_Binding 标识其实际执行平台；所指定的处理器或虚拟处理器承担该线程执行。

依据：A.1 Actual_Processor_Binding（第 242 页）。

A.1-04　内存候选按分类器筛选

存在有效 Allowed_Memory_Binding_Class 值时，从 Allowed_Memory_Binding 候选中只保留满足所列分类器的内存；没有该分类器约束时，保留全部允许内存候选。

依据：A.1 Allowed_Memory_Binding_Class（第 242 页）；13.2(C13)（第 223 页）。

A.1-05　代码与数据的实际内存

Actual_Memory_Binding 确定源程序代码和数据的实际内存位置；列表可包含多个内存，允许代码与数据等部分分别位于不同内存，不能把多项列表自动解释成每个内存均保存完整副本。

依据： A.1 Actual_Memory_Binding（第 243 页）。

A.1-06　分类器集合限制通信绑定候选

Allowed_Connection_Binding_Class 指定连接或虚拟总线可以采用的执行平台分类器集合，实际候选受该集合限制。

依据： A.1 Allowed_Connection_Binding_Class（第 243 页）。

A.1-07　特征分类器约束传播到途经连接

Allowed_Connection_Binding_Class 作用于特征时，所有经过该特征的连接都受到该通信平台分类器约束。

依据： A.1 Allowed_Connection_Binding_Class（第 243 页）。

A.1-08　实例集合限制通信绑定候选

Allowed_Connection_Binding 列出连接或虚拟总线可以采用的执行平台组件实例集合。

依据： A.1 Allowed_Connection_Binding（第 243 页）。

A.1-09　特征实例约束传播到途经连接

Allowed_Connection_Binding 作用于端口等特征时，所有经过该特征的连接都受到该通信平台实例集合的约束。

依据： A.1 Allowed_Connection_Binding（第 243 页）。

A.1-10　通信绑定列表保持路径次序

Actual_Connection_Binding 将连接或虚拟总线绑定到列出的总线、虚拟总线、处理器、虚拟处理器、设备或内存；列表项的先后顺序表示该连接经过执行平台的流路径顺序。

依据： A.1 Actual_Connection_Binding（第 244 页）。

A.1-11　功能与系统架构映射

Actual_Function_Binding 中的实例引用表示功能架构元素到系统架构元素的映射；它记录这种跨架构对应关系，不自动把它改写为处理器、内存或通信资源绑定。

依据： A.1 Actual_Function_Binding（第 244 页）。

A.1-12　远程调用候选物理路径

Allowed_Subprogram_Call_Binding 给出远程调用允许采用的执行平台物理连接；未指定时，调用可以是本地调用，也可从调用者和被调用者的绑定推断远程路径，不能仅据缺省断定所有调用本地化。

依据： A.1 Allowed_Subprogram_Call_Binding（第 244 页）。

A.1-13　远程调用实际物理路径

Actual_Subprogram_Call_Binding 指定远程调用实际绑定的总线、处理器、设备或内存；未指定时，调用可为本地调用，或其路径可由调用者与被调用者的绑定推断。

依据： A.1 Actual_Subprogram_Call_Binding（第 244 页）。

A.1-14　每个要求协议均须满足

Required_Virtual_Bus_Class 列表要求连接或虚拟总线所列的每个协议分类器都得到满足：可绑定到相应虚拟总线实例，也可绑定到 Provided_Virtual_Bus_Class 声明支持该协议的平台。它与 Allowed_Connection_Binding_Class 允许从所列分类器中选择的约束不同，不能把所需协议列表解释为任选一种协议。

依据： A.1 Required_Virtual_Bus_Class（第 245 页）；6.5(8)（第 111 页）；A.1 Provided_Virtual_Bus_Class（第 244–245 页）。

A.1-15　分离部署到不同硬件

Not_Collocated 要求 Targets 引用的软件元素使用 Location 所指类型的不同硬件；Location 为 system 时，不能把这些目标共置于该系统所包含的同一组件上。

依据： A.1 Not_Collocated（第 245 页）。

A.1-16　共置到同一硬件

Collocated 要求 Targets 引用的软件元素绑定到 Location 所指类型的同一硬件；Location 为 system 时，共置位置可以是该系统中包含的组件。

依据： A.1 Collocated（第 245 页）。

A.1-17　处理器支持的周期范围

Allowed_Period 给出处理器支持的周期区间集合；绑定线程的周期必须属于某个给定区间。未指定该属性时，不对这些线程的周期施加该项限制。

依据： A.1 Allowed_Period（第 247 页）。

A.1-18　总线物理访问分类器集合

Allowed_Physical_Access_Class 规定哪些处理器、设备、内存和总线分类器能够接入该总线；未指定时，总线可用于把设备和内存连接到处理器。

依据： A.1 Allowed_Physical_Access_Class（第 247 页）。

A.1-19　总线物理访问实例集合

Allowed_Physical_Access 的正式类型是组件 reference 列表，其值约束设备、处理器、内存和总线对该总线的物理接入；未指定时，总线可用于把设备和内存连接到处理器。

依据： A.1 Allowed_Physical_Access（第 247 页）。

A.1-20　内存协议决定可写能力

Memory_Protocol 默认为 read_write；源程序产生的可写数据只能放入 write_only 或 read_write 内存。execute_only 与 read_only 内存不承接这类可写数据。

依据： 6.3 Standard Properties（第 105 页）；A.1 Memory_Protocol（第 248 页）。

A.1-21　地址空间保护供给

Runtime_Protection_Support 表明处理器或虚拟处理器是否能在运行时强制保护地址空间；绑定到它的进程和虚拟处理器所声明的保护需求由该供给能力承接。

依据： A.1 Runtime_Protection_Support（第 248 页）。

A.1-22　调度策略由属性指定

Scheduling_Protocol 指定处理器线程调度器使用的策略，核心标准没有规定某个固定调度算法；所选策略可以协调多个处理器上的线程调度。

依据： A.1 Scheduling_Protocol（第 248 页）。

A.1-23　处理器线程容量

Thread_Limit 给出处理器或虚拟处理器支持的线程数上限；该值表示执行平台容量，不是对模型中声明线程的自动截断或选择。

依据： A.1 Thread_Limit（第 248 页）。

A.1-24　AADL 优先级映射到 RTOS

Priority_Map 按 Priority_Mapping 记录中的 Aadl_Priority 和 RTOS_Priority 配对，将 AADL 线程优先级映射到运行时操作系统优先级。

依据： A.1 Priority_Map 及 Priority_Mapping（第 248 页）。

A.1-25　处理器接受的优先级范围

Priority_Range 给出处理器或虚拟处理器可接受的线程优先级区间；它限制平台支持的优先级数值，核心标准不在此给定高值或低值的调度优先方向。

依据： A.1 Priority_Range（第 249 页）。

A.2　预声明线程属性

A.2-01　Dispatch_Able 控制是否分派

Dispatch_Able 指定线程应否被分派，可随线程自身模式决定活动或空闲；不因线程仍有声明便忽略该可分派开关。

依据： A.2 Dispatch_Able（第 249 页）。

A.2-02　零计算预算不关闭分派

Compute_Execution_Time 为零仍表示线程被分派、由应用决定本次没有工作，不等同于 Dispatch_Able 禁止分派。

依据： A.2 Dispatch_Able（第 249 页）。

A.2-03　POSIX 同优先级 FIFO 调度

采用 POSIX 调度且 POSIX_Scheduling_Policy 为 SCHED_FIFO 时，在相同固定优先级集合内按 FIFO 执行，线程保持处理器直至完成或更高优先级线程释放；未定义策略时默认采用 SCHED_FIFO。

依据： A.2 POSIX_Scheduling_Policy（第 249–250 页）。

A.2-04　POSIX 同优先级轮转调度

POSIX_Scheduling_Policy 为 SCHED_RR 时，除 FIFO 方式外，quantum 耗尽还会抢占当前线程，使同优先级线程获得运行机会。

依据： A.2 POSIX_Scheduling_Policy（第 249–250 页）。

A.2-05　POSIX 其他策略由实现定义

选择 POSIX 的 SCHED_OTHER 策略时，具体执行行为由该策略实现者定义，不能从名称推定固定轮转算法。

依据： A.2 POSIX_Scheduling_Policy（第 249–250 页）。

A.2-06　线程优先级的数值方向

调度协议采用线程 Priority 时，数值较大表示优先级较高；未使用 Priority 的协议不因该属性存在而自动改变调度次序。

依据： A.2 Priority（第 250 页）。

A.2-07　数据并发优先级的数值方向

并发控制协议采用 data 或 data access 的 Priority 解决冲突时，数值较大表示较高访问优先级。

依据： A.2 Priority（第 250 页）。

A.2-08　静态时隙分配

采用静态时隙调度的协议时，Time_Slot 给出分配给组件的时间线槽位，调度以这些已分配槽位为依据。

依据： A.2 Time_Slot（第 250 页）。

A.2-09　未指定并发协议时不自动加锁

Concurrency_Control_Protocol 未关联值时默认 None_Specified，不由该缺省值隐式添加互斥协议。

依据： A.2 Concurrency_Control_Protocol（第 250 页）。

A.2-10 Dequeued_Items 限制冻结可见数量

AllItems 或 MultipleItems 端口设置 Dequeued_Items 时，一次输入冻结向应用提供的项数不超过该值；该上限不大于同端口的 Queue_Size。

依据： A.2 Dequeue_Protocol、Dequeued_Items（第 251 页）。

A.2-11 unload 停用策略重新装载

进程 Deactivation_Policy=unload 时，在停用时卸载，并在重新激活时重新装载。

依据： A.2 Deactivation_Policy（第 252 页）。

A.2-12 inactive 停用策略保持装载

进程 Deactivation_Policy=inactive（默认）时，在启动时装载，模式停用不卸载该进程。

依据： A.2 Deactivation_Policy（第 252 页）。

A.2-13 线程停用策略描述状态保存

Deactivation_Policy 作用于 thread 时描述线程状态是否保存；不能将进程的卸载动作直接当作线程状态表示。

依据： A.2 Deactivation_Policy（第 252 页）。

A.3 预声明时间属性

A.3-01 入口执行时间的计费范围

Initialize、Activate、Compute、Deactivate、Finalize 及 Recover 的 Execution_Time 包含本线程执行的服务调用开销，排除其他线程响应其远程调用消耗的执行时间。

依据： A.3 Activate_Execution_Time（第 253 页）；A.3 Compute/Deactivate/Finalize/Initialize/Recover_Execution_Time（第 253–257 页）。

A.3-02 计算与恢复期限受总期限约束

给定线程 Deadline 时，Compute_Deadline + Recover_Deadline 不得超过该 Deadline；这是预算之和的约束，即使本次实际较早完成也须成立。

依据： A.3 Compute_Deadline（第 253 页）；A.3 Deadline、Recover_Deadline（第 254、256 页）。

A.3-03 分派总经过时间期限

非 Background 线程从分派到重新开始等待下一分派的经过时间不得超过有效 Deadline；该经过时间包含本次计算、等待及适用恢复。

依据： A.3 Deadline（第 254 页）。

A.3-04　Deadline 缺省引用当前 Period

Deadline 未取得更优先属性关联值时，采用其默认表达式 Period，在当前线程实例上下文求值。

依据： A.3 Deadline（第 254 页）；11.4(8)（第 210 页）。

A.3-05　第一次分派请求的时刻

First_Dispatch_Time 指定第一次分派请求的时刻；该值不直接指定线程首次取得处理器的时刻。

依据： A.3 First_Dispatch_Time（第 254 页）。

A.3-06　分派请求迟延的上限

Dispatch_Jitter 给出分派请求相对预定时刻允许的最大迟延；它不计线程已经分派后等待处理器的调度延迟。

依据： A.3 Dispatch_Jitter（第 254–255 页）。

A.3-07　虚拟处理器的执行时间预算

Execution_Time 是物理处理器分配给所绑定虚拟处理器的执行时间，也是该虚拟处理器可继续提供给所调度线程或下层虚拟处理器的执行时间。

依据： A.3 Execution_Time（第 255 页）。

A.3-08　系统 Load_Time 计量共享数据装载

Load_Time 作用于 system 时，计量系统实现中跨进程共享 data 二进制映像的装载时间。

依据： A.3 Load_Time（第 256 页）。

A.3-09　恢复期限从检出错误起计算

非 Background 线程的 Recover_Deadline 限定从检出错误到开始等待下一分派的最大经过时间；排队等待恢复执行也消耗该期限。

依据： A.3 Recover_Deadline（第 256 页）。

A.3-10　恢复期限不超过已给周期

非 Background 线程已指定 Period 时，其 Recover_Deadline 不得大于该 Period。

依据： A.3 Recover_Deadline（第 256 页）。

A.3-11　跨处理器时钟中断抖动

Clock_Jitter 限定多处理器系统任意两个处理器开始处理同一时钟中断的最大时间差；比较的是开始处理时刻。

依据： A.3 Clock_Jitter（第 257 页）。

A.3-12　相邻时钟中断的周期

Clock_Period 确定相邻时钟中断之间的时间间隔。

依据： A.3 Clock_Period（第 257 页）。

A.3-13　可配置时钟周期的范围

Clock_Period_Range 给出可赋给 Clock_Period 的下、上界，不把该范围解释为每次中断均可任意抖动。

依据： A.3 Clock_Period_Range（第 257 页）。

A.3-14　跨进程线程切换时间

Process_Swap_Execution_Time 给出不同进程间线程上下文切换所需执行时间范围；无运行时错误时采用该范围计量。

依据： A.3 Process_Swap_Execution_Time（第 258 页）。

A.3-15　调度时间片上限

使用 Scheduler_Quantum 的调度器以该值限制线程连续占用处理器而不被抢占的最长时间；处理器未指定该属性时，附录规定其为正无穷小值。SCHED_RR 等采用时间片的策略据此轮换执行。

依据： A.3 Scheduler_Quantum（第 258 页）。

A.3-16　同进程线程切换时间

Thread_Swap_Execution_Time 给出同一进程内线程上下文切换所需执行时间范围；无运行时错误时采用该范围计量。

依据： A.3 Thread_Swap_Execution_Time（第 258 页）。

A.3-17　静态主帧周期

采用主帧的静态调度协议时，Frame_Period 给出该主帧重复的时间周期。

依据： A.3 Frame_Period（第 258 页）。

A.3-18　静态固定时隙长度

静态调度协议采用固定时隙时，Slot_Time 给出每个时隙的时间长度；它不同于 Time_Slot 指定的槽位编号。

依据： A.3 Slot_Time（第 259 页）。

A.4　预声明通信属性

A.4-01　数组连接缺省采用 One_To_One

未设置 Connection_Pattern 时，其有效值取 One_To_One。

依据： A.4 Connection_Pattern（第 259 页）。

A.4-02　pull 数据按接收者速率传输

data 端口连接采用 Transmission_Type=pull 时，实际数据传输按接收者的速率进行，不按源每次产生数据都额外形成传输。

依据： A.4 Transmission_Type（第 260 页）。

A.4-03　事件 pull 等待接收者请求

event 或 event data 通信采用 Transmission_Type=pull 时，内容在发送者处排队，收到接收者请求才传输，不仅因源产生事件就按 push 交付。

依据：A.4 Transmission_Type（第 260 页）。

A.4-04　总线发送计时不含仲裁和排队

Transmission_Time 计量首 bit 上 bus 至末 bit 上 bus 的时间，不包含仲裁、排队或其他由 bus 竞争和调度产生的等待。

依据：A.4 Transmission_Time（第 262 页）。

A.4-05　总线发送耗时按线性模型求值

N Bytes 消息的 bus 发送耗时为 F+N×P，其中 F 与 P 分别从 Transmission_Time.Fixed 和 PerByte 的非负时间范围取值。

依据：A.4 Transmission_Time（第 262 页）。

A.4-06　Actual_Latency 表示实现路径的实际延迟

Actual_Latency 表示由实现沿语义连接确定的实际连接或端到端 flow 延迟范围。

依据：A.4 Actual_Latency（第 262 页）。

A.4-07　Latency 约束完整进入离开耗时

给定 Latency 范围时，比较数据或事件从进入连接或 flow 到离开的完整经过时间与该下、上界；不能仅用不含排队仲裁的 bus 发送时间代替该实际经过时间。

依据：A.4 Latency（第 262 页）。

A.5　预声明内存属性

A.5-01　只读访问的读写效果

Access_Right=read_only 的访问允许读取被访问数据，不允许通过该访问写入。

依据：A.5 Access_Right、Access_Rights（第 263 页）。

A.5-02　只写访问的读写效果

Access_Right=write_only 的访问允许写入被访问数据，不允许通过该访问读取。

依据：A.5 Access_Right、Access_Rights（第 263 页）。

A.5-03　读写访问及缺省权限

Access_Right=read_write 允许直接读取和写入；未指定 Access_Right 时，其默认值为 read_write。

依据：A.5 Access_Right、Access_Rights（第 263 页）。

A.5-04　方法访问的实现

Access_Right=by_method 时，通过数据类型提供的子程序间接访问数据，而非由该权限赋予直接字段读写。

依据： A.5 Access_Right、Access_Rights（第 263 页）。

A.5-05　提供访问受组件权限限制

provides access 所提供的权限不得超过被访问组件自身的 Access_Right 权限上限。

依据： A.5 Access_Right、Access_Rights（第 263 页）。

A.5-06　要求访问受提供权限限制

requires access 的有效访问需求不得超过提供访问方及被访问组件本身允许的权限。

依据： A.5 Access_Right、Access_Rights（第 263 页）。

A.5-07　过短消息填充至允许大小

数据消息小于 Allowed_Message_Size 下限时必须填充；该区间约束单次传输的数据块大小，不计协议头或分包开销。

依据： A.5 Allowed_Message_Size（第 264 页）。

A.5-08　过长消息分包传输

数据消息大于 Allowed_Message_Size 上限时必须拆成两个或更多独立传输的包；该上限针对数据块，不包括协议头及分包开销。

依据： A.5 Allowed_Message_Size（第 264 页）。

A.5-09　处理器字节搬移时间

在指定处理器上搬移 N 字节的估计执行时间由 Assign_Time 给出，为 Fixed + N × PerByte；Fixed 和 PerByte 分别采用其属性范围内的值。

依据： A.5 Assign_Time（第 264 页）。

A.5-10　Base_Address 定位内存首字

Base_Address 作用于 memory 组件时，给出该内存首个字的起始地址。

依据： A.5 Base_Address（第 264 页）。

A.5-11　字地址按 Word_Space 递增

内存第 k 个后继字地址为 Base_Address+k×Word_Space；Word_Space 默认 1，取大于 1 的值时相邻字使用不连续的整数地址。

依据： A.5 Base_Address（第 264 页）；A.5 Word_Space（第 266 页）。

A.5-12 Base_Address 定位绑定对象

Base_Address 作用于 data、data access 或 port 时，给出该对象在其实际绑定内存中的起始地址。

依据： A.5 Base_Address（第 264 页）。

A.5-13 设备端口寄存器地址

Device_Register_Address 存在时，将对应设备端口定位到指定设备寄存器地址；该属性可省略，设备端口也可以由设备驱动源码中的变量表示。

依据： A.5 Device_Register_Address（第 264 页）。

A.5-14 内存读取时间

从内存读取 N 字节的估计执行时间由 Read_Time 确定，为 Fixed + N × PerByte；两项分别取其属性指定范围内的值。

依据： A.5 Read_Time（第 264 页）。

A.5-15 代码空间包含只读数据

Code_Size 计入相关源码在最终系统中完成编译、链接、绑定和装载后的静态代码及只读数据大小。

依据： A.5 Code_Size（第 265 页）。

A.5-16 可写数据空间大小

Data_Size 计入相关源码在最终系统中形成的可读写数据大小。

依据： A.5 Data_Size（第 265 页）。

A.5-17 堆需求为最大容量

Heap_Size 给出线程或子程序的最大堆空间需求，而非某时刻已分配或平均占用的堆大小。

依据： A.5 Heap_Size（第 265 页）。

A.5-18 栈需求为执行期最大值

Stack_Size 给出处理器执行支持软件、设备驱动、线程或子程序执行时所使用栈的最大大小。

依据： A.5 Stack_Size（第 265 页）。

A.5-19 内存资源容量

Memory_Size 给出 memory、processor、virtual processor 或 system 所表示的内存大小。

依据： A.5 Memory_Size（第 265 页）。

A.5-20 可独立读写的最小存储单位

Word_Size 定义内存中可独立读写的最小存储单位大小，默认 8 bits；其大小不等同于地址步长，地址步长由 Word_Space 指定。

依据： 6.3 Standard Properties（第 105 页）；A.5 Word_Size（第 265 页）。

A.5-21　内存写入时间

向内存写入 N 字节的估计执行时间由 Write_Time 确定，为 Fixed + N × PerByte；两项分别取其属性指定范围内的值。

依据： A.5 Write_Time（第 266 页）。

A.6　预声明编程属性

A.6-01　调用序列入口的函数映射

使用 Compute、Deactivate、Finalize、Initialize 或 Recover 的 Entrypoint_Call_Sequence 时，由无参数入口函数执行所引用调用序列并传递相应端口及参数实际值；该函数允许由模型生成。

依据： A.6 Compute_Entrypoint_Call_Sequence（第 267 页）；A.6 Deactivate_Entrypoint_Call_Sequence（第 268 页）；A.6 Finalize_Entrypoint_Call_Sequence、Initialize_Entrypoint_Call_Sequence（第 269 页）；A.6 Recover_Entrypoint_Call_Sequence（第 270 页）。

A.6-02　未指定语言允许推断或采用工具缺省

组件未指定 Source_Language 时，处理方法可以从源文件路径语法推断语言，也可以设置自身默认语言；核心标准没有给出统一默认源语言。

依据： A.6 Source_Language（第 271 页）。

A.6-03　平台 Source_Language 描述平台软件

processor 或 virtual processor 上的 Source_Language 描述实现其平台功能的软件语言，而非自动改写所有被绑定应用组件的语言。

依据： A.6 Source_Language（第 271 页）。

A.6-04　源文件路径的定位

Source_Text 各项按 POSIX 路径语义解释，工具可以支持环境变量、模式等扩展。路径首字符为点时，相对于存放该 AADL 规格文本的文件目录；否则默认定位目录由处理方法决定。

依据： A.6 Source_Text（第 271 页）。

A.6-05　全部源文件构成关联源文本

Source_Text 列表中全部文件的内容共同构成该关联的源文本集合，不能只采用列表首项；该属性没有标准默认文件集合。

依据： A.6 Source_Text（第 271 页）。

A.6-06　平台支持语言与缺省无限制

Supported_Source_Language 列出处理器支持的源码语言；任何绑定线程可能访问的软件组件，其语言须属于该集合。未指定支持列表时，没有这项语言限制。

依据： A.6 Supported_Source_Language（第 271–272 页）。

A.6-07　硬件描述路径的定位

Hardware_Description_Source_Text 的文件路径按 POSIX 语义解释并允许工具扩展；首字符为点时相对于包含 AADL 规格文本的文件目录，否则默认定位目录由处理方法确定。

依据：A.6 Hardware_Description_Source_Text（第 272 页）。

A.6-08　硬件描述语言由属性指定

Hardware_Source_Language 指定关联硬件描述源文本所采用的硬件描述语言。

依据：A.6 Hardware_Source_Language（第 273 页）。

A.6-09　显式设备驱动模型引用

Device_Driver 引用一个 abstract implementation，其中包含驱动子程序实例或中断服务线程 ISR 实例；该引用指出设备驱动的显式实现结构。

依据：A.6 Device_Driver（第 273 页）。

A.8　项目属性集

A.8-01　RMS 按周期线程速率分配优先级

采用 RMS 调度时，周期线程按速率递减顺序分配优先级。

依据：A.8 Supported_Scheduling_Protocols（第 276 页）。

A.8-02　RMS 按最小到达间隔安排事件线程

采用 RMS 调度时，aperiodic 及 sporadic 线程按其最小到达间隔安排。

依据：A.8 Supported_Scheduling_Protocols（第 276 页）。

A.8-03　RMS 按 FIFO 安排 Background 线程

采用 RMS 调度时，Background 线程按 FIFO 安排。

依据：A.8 Supported_Scheduling_Protocols（第 276 页）。

附录文档 F（本卷收录） Data Sets

F.1 Data Sets 的定义与关联

F.1-01 同一模型可附加多个 DataSet

在目标组件实现的扩展中通过 DataSet 子句引用 dataset 库中定义的数据集，可向同一模型附加不同属性数据集及其组合。

依据： Annex F.1(2),(L1)（第 337 页）。

F.1-02 DataSet 的包含路径从声明目标实现出发

DataSet 中 contained 属性关联的 applies to 路径，其共同路径根为该 dataset 定义的 applies to 语句所引用的组件实现。

依据： Annex F.1(N3)（第 337 页）。